

ModelFlow - Model Specification

October 2, 2026

Contents

1	Introduction	1
1.1	What is in this book	1
1.2	Prerequisites	1
2	Modelflow brings together a model and a Pandas DataFrame	4
2.1	Specification of a model	4
2.2	Example Models	5
2.3	A Model	6
2.4	Why Separate Specification and Implementation?	6
2.5	A Simple Specification Language	7
2.6	Makemodel – Expanding the Stripped-Down DSL	7
2.7	Lists used in the next examples	11
2.8	Loop over all banks	12
2.9	Sum of variables	13
2.10	Combining doable and sum	14
2.11	Tags and Formula Decoration	15
2.12	Using <code>replacements=</code> or <code>str.replace()</code> to simplify multi-dimensional formulas . . .	17
2.13	Using <code>list_defs=</code> to define lists in Makemodel	19
2.14	Using <code>functools.partial</code> to create simplified Makemodel constructors	20
2.15	Building modular models by combining smaller formula groups	20
2.16	Estimating behavioural equations	26
2.17	<code>%%Makemymodel</code> — a Jupyter magic wrapper for Makemodel	29
3	The <code>%%Makemymodel</code> Jupyter Magic	32
3.1	1. Basic syntax	32
3.2	2. Available options	33
3.3	3. Using lists and <code>doable</code> inside the magic	34
3.4	4. Segmented models	34
3.5	5. Using <code>replacements=</code> for dimension placeholders	36
3.6	6. Estimation inside the magic	37
3.7	7. Tags inside the magic	38
3.8	8. Generating a LaTeX PDF	39
3.9	9. How it works internally	39
3.10	10. Quick reference	39
4	Estimating behavioural equations	40
4.1	A first example, where the answer is known	40
4.2	A linear consumption function	41
4.3	Consumption as an error correction equation	42
4.4	A named estimator with <code>.with_defaults()</code>	44
4.5	The full syntax of an estimation	45
4.6	Constraints — the <code>ST.</code> clause	46
4.7	Options reference	47

4.8	Estimation while building a model	48
4.9	Quick reference	48
5	The normal function	50
5.1	Imports	50
5.2	The result class: <code>Normalized_frm1</code>	50
5.3	The signature of <code>normal</code>	51
5.4	Basic normalization	51
5.5	Add factor, fixing and fitted equation combined	52
5.6	Built-in transformation functions on the right-hand side	52
5.7	Transformations on the left-hand side	53
5.8	A real-world example	55
5.9	Leads and the <code>EViews D()</code> operator	56
5.10	Choosing the endogenous variable	56
5.11	Implicit equations	56
5.12	Summary: the preprocessing steps	57
6	When using <code>Makemodel</code> og <code>%%Makemymodel</code> the tag <code><add></code> will introduce an add factor	58
6.1	Activate the <code>%%Makemymodel</code> magic cell	58
6.2	A simple equation	58

Chapter 1

Introduction

This supplement is about **how a model is specified in ModelFlow** — the step that turns a piece of readable text into a model object that can be simulated on a Pandas DataFrame.

A ModelFlow model is written in a small domain specific language (DSL). Each equation states a piece of business logic, one endogenous variable per equation:

```
| > profit_{banks} = revenue_{banks} - expenses_{banks}
```

Before such text can be solved, it has to be **expanded** (loops over lists, sums, tags), **normalized** (so the endogenous variable stands alone on the left-hand side) and optionally extended with **add factors**. **Makemodel** performs the expansion, **normal** performs the normalization, and the **%%Makemodel** cell magic lets you do the whole thing in a notebook cell.

1.1 What is in this book

Model specification introduces the DSL and the **Makemodel** preprocessor: list definitions and **do-loops**, **doable**, **sum()**, the **<exo>**, **<add>** and **<fit>** tags, **replacements=** and **list_defs=**, and how several model blocks are combined with **+** into a **Listmodels** container.

The %%Makemodel magic shows the notebook front end: writing Markdown documentation and **>** equations in the same cell, the available options (**show**, **draw**, **render**, **latex**), and how **segment=** lets you build a large model from independently developed pieces.

Estimation covers the behavioural equations, whose coefficients come from data rather than from accounting: the equation syntax for coefficients and constraints, the **ols** and **nls_lmfit** backends, and how an equation tagged **<estimator=...>** is estimated while the model is being built. It stays with the syntax, demonstrated on small synthetic data where the right answer is known in advance; a complete estimation on real data is a supplement of its own, *ModelFlow - Estimation, a worked example*.

Normalization goes one level down, to the **normal** function that rewrites an equation into $y = F(y, x)$ form, and to the **Normalized_frml** class that carries the original, preprocessed, normalized and fitted versions of an equation.

Add factors looks in detail at what the **<add>** tag actually generates, and how the resulting add factor can be used to control an equation.

1.2 Prerequisites

The notebooks import from the ModelFlow source, mainly:

```
| from modelconstruct_estimation import Makemodel
| import modeljupytermagic           # registers the %%Makemodel magic
| from modelnormalize import normal
```

Estimation needs no import of its own: the method is named in the equation's tag, as `<est=ols>` or `<est=nls_lmfit>`.

Each notebook can be read on its own, but the chapters are ordered so that later ones build on the vocabulary of the earlier ones.

Chapter 2

Modelflow brings together a model and a Pandas DataFrame

ModelFlow is designed to combine a **model specification** with a **Pandas DataFrame** that contains the corresponding data.

In this setup, each **column** of the DataFrame represents a model variable, while the **row index** represents time.

This structure allows ModelFlow to evaluate and simulate the model directly on the data, period by period.

While Pandas provides powerful tools for managing time series, it does not natively handle **lagged variables** or **simultaneous equation systems** in a convenient way.

ModelFlow fills this gap by extending the DataFrame framework with capabilities for **lag and lead referencing**, **recursive and simultaneous solution algorithms**, and **automatic dependency handling** between variables.

In this way, ModelFlow bridges the gap between **data storage** and **model computation**, making it possible to define, solve, and analyze complex dynamic systems directly within the familiar Python and Pandas environment.

```
from modelconstruct_estimation import Makemodel
import modeljupytermagic
```

2.1 Specification of a model

In its simplest form, the ModelFlow domain-specific language (DSL) consists of **formulas**, each defining a relationship between variables.

However, ModelFlow also supports a **higher-level DSL** that extends this basic syntax with Python-based text processing features.

This enables the user to **generate formulas programmatically**, allowing the model specification itself to be written at a **higher level of abstraction**.

With this extended DSL, users can define **templates or patterns** for equations that apply across multiple entities — for example, all sectors in a macroeconomic model or all banks in a stress-testing framework — and then automatically expand them into a complete set of detailed, entity-specific equations.

This approach dramatically reduces **boilerplate text** and improves **consistency** across the model. Instead of manually writing large blocks of repetitive equations, the modeler can rely on Python constructs such as loops, lists, and string formatting to generate the full specification dynamically and transparently.

By integrating this higher-level DSL directly into the modeling framework, ModelFlow combines the **clarity and expressiveness** of a concise formula language with the **flexibility and automation** capabilities of Python — making it possible to specify large, structured models in a compact, readable, and reproducible way.

2.2 Example Models

To illustrate the use of ModelFlow and its formula-based DSL, we present two simple examples.

The first is a small macroeconomic model describing the relationship between **GDP** and **consumption**.

The second is a **debt dynamics** model that tracks the evolution of government debt over time.

Together, they demonstrate how behavioral and accounting identities can be expressed concisely in ModelFlow's syntax.

2.2.1 1. GDP and Consumption Model

This simple model links output Y_t and consumption C_t over time.

Consumption depends on lagged disposable income, and output is determined by aggregate demand.

$$\begin{aligned} C_t &= \alpha_0 + \alpha_1 Y_t + \alpha_2 Y_{t-1} + \varepsilon_t^C \\ Y_t &= C_t + I_t + G_t + X_t - M_t \end{aligned}$$

- C_t – consumption at time t
- Y_t – output (GDP) at time t
- I_t, G_t, X_t, M_t – investment, government spending, exports, and imports
- $\alpha_0, \alpha_1, \alpha_2$ – behavioral parameters
- ε_t^C – consumption shock

This captures **lagged adjustment behavior** (consumption depends on past income) while treating the other demand components as **exogenous**. Also the model contains **contemporaneous feedback loops** so it has to be solved as a system.

To input this into ModelFlow the two formulas can be written as:

```
Frml <stoc> C = a0 + a1*Y + a2*Y(-1) + epsC $
Frml Y = C + I + G + X - M $
```

2.2.2 2. Debt Dynamics Model

A second example tracks the evolution of government debt over time.

Debt increases with the primary deficit and with interest payments on existing debt.

$$\begin{aligned} D_t &= (1 + i_{t-1})D_{t-1} + PD_t \\ PD_t &= G_t - T_t \end{aligned}$$

- D_t – government debt at time t
- i_{t-1} – interest rate on debt from the previous period
- PD_t – primary deficit (spending minus taxes)
- G_t – government spending
- T_t – taxes

Here, D_t is **endogenous**, while i_t , G_t , and T_t may be treated as **exogenous inputs** or determined by other model blocks.

This simple dynamic ensures that current debt depends on **lagged debt** and **current fiscal flows**, a common structure in fiscal and macroeconomic models.

To input this into ModelFlow the two formulas can be written as:

```
Frml D = (1 + i(-1))*D(-1) + PD $
Frml PD = G - T $
```

2.3 A Model

For the purpose of this note, a (simple) model like the two mentioned above can be thought of as a system of equations that links **endogenous** and **exogenous** variables over time.

- **Endogenous variables** are those determined *within* the model — their values are the outcome of the relationships the model describes.
- **Exogenous variables**, on the other hand, are determined *outside* the model — they are treated as given inputs or shocks that influence the endogenous variables but are not themselves affected by them.

In time-based models, variables can appear with **lags** or **leads**.

A *lag* refers to the value of a variable in a previous period — for example, y_{t-1} is the value of y one period before t .

A *lead* refers to a future value — for example, y_{t+1} represents the value of y one period after t .

Including lags allows the model to capture persistence and delayed effects in economic relationships, such as habit formation or gradual adjustment.

In this note, only **lags** will be used, as they are sufficient for representing the backward-looking relationships typical of most empirical and accounting-based models.

However, **ModelFlow** also allows the use of *leaded* variables, which are useful in models with rational or model-consistent expectations.

Lets say we have:

- **n**: number of endogenous variables
- **k**: number of exogenous variables
- **r**: maximum lag of endogenous variables
- **s**: maximum lag of exogenous variables
- **t**: time index (year, quarter, month, day, or any other unit)

$$\begin{aligned}
 y_t^1 &= f^1(y_t^2, \dots, y_t^n, \quad y_{t-1}^1, \dots, y_{t-r}^n, \quad x_t^1, \dots, x_t^k, \quad x_{t-1}^1, \dots, x_{t-s}^k) \\
 y_t^2 &= f^2(y_t^1, \dots, y_t^n, \quad y_{t-1}^1, \dots, y_{t-r}^n, \quad x_t^1, \dots, x_t^k, \quad x_{t-1}^1, \dots, x_{t-s}^k) \\
 &\vdots \\
 y_t^n &= f^n(\underbrace{y_t^1, \dots, y_t^{n-1}}_{\text{current}}, \underbrace{y_{t-1}^1, \dots, y_{t-r}^n}_{\text{lagged}}, \underbrace{x_t^1, \dots, x_t^k}_{\text{current}}, \underbrace{x_{t-1}^1, \dots, x_{t-s}^k}_{\text{lagged}}) \\
 &\qquad\qquad\qquad \underbrace{\hspace{10em}}_{\text{endogenous}} \hspace{1em} \underbrace{\hspace{10em}}_{\text{exogenous}}
 \end{aligned}$$

Sometime such models are implemented directly in a programming environment such as Python, Matlab, or even Excel, where they are both *specified* and *solved* in the same code.

2.4 Why Separate Specification and Implementation?

Instead of mixing economic reasoning with programming details, a model can be written in a **domain-specific language (DSL)** designed specifically for economics and finance. Separating specification from implementation has several advantages:

1. **Clarity and focus** – The specification remains close to the underlying economic logic, expressed in a concise and readable form. The modeler concentrates on *what the relationships are*, not on *how the computer solves them*.
2. **Expressiveness** – The DSL uses syntax familiar to economists (equations, lags, variables), avoiding the clutter of general-purpose programming constructs.

3. **Maintainability** – Models can be extended or updated without rewriting solution code. Changing an equation is as simple as editing one line in the specification.
4. **Portability** – Once specified, the same model can be executed with different numerical engines or platforms without altering its economic content.
5. **Transparency** – The separation makes models easier to review, validate, and communicate, since the specification reads like economic theory rather than computer code.

This approach is common in several econometric systems such as **EViews**, **Dynare**, and **GEKKO**, which all use their own DSLs to describe models.

ModelFlow follows a similar philosophy but with a key difference: it does not introduce a new language for data handling, visualization, or model analysis. Instead, ModelFlow relies on the **Python ecosystem** - primarily **pandas**, but also **networkx**, **sympy**, **numpy**, and other libraries - allowing the model specification to be combined seamlessly with powerful tools for data manipulation, result analysis, and visualization.

2.5 A Simple Specification Language

In practice, the DSL for economic models is built around equation-like statements. For example:

```
Frml <stoc> C = a0 + a1*Y(-1) + epsC $
Frml Y = C + I + G + X - M $
```

Here each formula (**Frml**) defines the relationship between variables in a way that mirrors how economists usually write them, with lags (**(-1)**) and simple algebraic notation.

The optional **<tag>** is used for transforming the **frml** or to control solving - more on that later.

This kind of business-logic-oriented specification acts as the “blueprint” of the model, leaving the task of solving it to the underlying implementation engine.

2.6 Makemodel – Expanding the Stripped-Down DSL

The basic **Frml** statements form the **lowest level** of the ModelFlow domain-specific language (DSL), defining the direct relationships between variables. The basic **Frml** statements are transpiled into Python when the model is solved.

To make the language more expressive, concise, and suitable for large, structured models, ModelFlow uses a preprocessor called **Makemodel** to extend this foundation with several higher-level constructs:

- **Lists and sublists** – to organize entities such as banks, sectors, or countries, and automatically expand equations across these groups.
- **Do ... enddo loops** – to generate sets of equations programmatically over the members of a list or over a specified range.
- **Sum operator** – to express aggregation directly within equations by summing over lists or sublists.
- **Doable equations** – shorthand notation combining loops and summation, simplifying the definition of repeated or aggregated relationships.
- **Stripped-down syntax** – allows equations to be written without explicit **Frml** and **\$** markers, resulting in cleaner, more compact specifications.
- **Model specifications embedded in Markdown** – each line of model code starts with **>**, making them visually distinct and easily readable within documents. Documentation and explanations can be formatted nicely alongside the code using Markdown.

Together, these features make it possible to write models at a **higher level of abstraction**, reducing repetition and improving consistency across model components.

Instead of defining each equation manually, the modeler can use lists, loops, and operators to express

economic logic once and have ModelFlow automatically generate all the necessary detailed equations. This design allows the DSL to scale from small illustrative examples to large, multi-entity macroeconomic and financial models while maintaining clarity and transparency.

2.6.1 Makemodel Properties

Box 1 Makemodel Properties

For use by the very interested modeller, the `Makemodel` dataclass stores the full transformation state of a model as it moves from the stripped-down DSL to the fully expanded, normalized set of `Frml` statements ready for ModelFlow.

Each field corresponds to a specific stage in this expansion and normalization process.

Property	Type	Description
<code>original_statements</code>	<code>str</code>	Input model specification written in the stripped-down DSL.
<code>normal_frml</code>	<code>str</code>	Output containing the normalized model equations.
<code>normal_main</code>	<code>str</code>	Normalized <code>Frml</code> statements for the main model equations.
<code>normal_fit</code>	<code>str</code>	Normalized <code>Frml</code> statements for fitted-value calculations.
<code>normal_calc_add</code>	<code>str</code>	Normalized <code>Frml</code> statements for add-factor calculations.
<code>clean_frml_statements</code>	<code>str</code>	Intermediate version with properly formatted <code>Frml</code> lines and structured list syntax.
<code>post_doable</code>	<code>str</code>	Model after expansion of all <code>doable</code> equations.
<code>post_do</code>	<code>str</code>	Model after expansion of explicit <code>do ... enddo</code> loops.
<code>post_sum</code>	<code>str</code>	Model after expansion of all summation (<code>sum</code>) operators.
<code>expanded_frml</code>	<code>str</code>	Fully expanded set of executable <code>Frml</code> equations.
<code>normal_expressions</code>	<code>List[Any]</code>	List of parsed normalized expression objects.
<code>list_specification</code>	<code>str</code>	String representation of all list definitions found in the model.
<code>modellist</code>	<code>str</code>	Dictionary-like string of defined lists and their attributes.
<code>funks</code>	<code>List[Any]</code>	List of user-defined functions that can be referenced in model formulas.

These internal fields allow the modeller to follow, in detail, how the stripped-down DSL is gradually transformed into executable ModelFlow syntax.

They provide a complete diagnostic view of the intermediate stages—useful for debugging, educational purposes, or extending ModelFlow’s preprocessing pipeline.

The higher-level properties such as `show` and `mmodel`—described in the main `Makemodel` section—build on this internal structure to present the final expanded model or to produce a fully functional ModelFlow model instance.

2.6.2 Inspecting a property with the `show` prefix

Every one of those properties can be printed with a **show prefix**. `Mtoymodel.normal_frml` returns the string; `Mtoymodel.shownormal_frml` prints it laid out, which for a multi-line block of `Frml` statements is the difference between something readable and one long line with `\n` in it.

The **show** properties are the ordinary way to look at a model under construction:

Property	Shows
<code>showclean_frml_statements</code>	the equations as <code>Frml</code> statements, without estimated parameter values
<code>shownormal_frml</code>	the normalized equations <i>with</i> estimated values, and with the generated <code>_A</code> , <code>_X</code> and <code>_D</code> variables
<code>shownormal_main</code> , <code>shownormal_calc_add</code> , <code>shownormal_fit</code>	the same, split into main equations, add-factor calculations and fitted values
<code>showexpanded_frml</code> , <code>showpost_do</code> , <code>showpost_sum</code> , <code>showpost_doable</code>	the intermediate stages of expansion, in order
<code>showlists</code> , <code>showlist_specification</code>	the lists the model defines
<code>showestimation_records</code>	what happened in each estimation run during construction
<code>showvar_description</code> , <code>showsmpl</code> , <code>showestimator</code>	the options the model was built with

The prefix is generated rather than written out, so it works for any field of the dataclass, including any added later. Asking for one that does not exist is the quickest way to see the whole list: `<model>.showclean` raises an error which prints every available **show** property, each with the description of the field behind it.

A model built and inspected this way is in the *ModelFlow - Estimation, a worked example* supplement.

2.6.3 Lists

Among the higher-level constructs, **lists** and **sublists** are central to making ModelFlow scalable and expressive.

They allow the modeler to define **groups of entities** (such as banks, portfolios, sectors, or countries) and to attach **attributes** or **hierarchies** to those entities.

This enables equations to be written once as templates and then **expanded automatically** across all relevant list members.

Sublists can be used to define logical relationships between entities — for example, linking each bank to its home country, or associating portfolios with regulatory categories.

These structures make it possible to organize models systematically and to maintain consistency across entities that share the same behavioral equations.

This approach eliminates redundancy and manual repetition: instead of writing dozens or hundreds of similar equations, the modeler defines a **single equation pattern**, and ModelFlow expands it dynamically over the members of a list.

This capability is especially valuable in:

- **Stress-testing models** – where many banks, each with multiple portfolios, must be represented consistently.
- **Macroeconomic models** – where sectors of the economy (households, firms, government, rest of the world, etc.) share common behavioral structures.
- **Cross-country or panel models** – where the same specification is applied across multiple countries or regions.

Syntax

List definitions in the ModelFlow DSL follow a simple and consistent format:

- Each list **starts** with the keyword `list`.
- It can contain one or more **sublists**, separated by `/`.

- Each sublist **starts** with a **name**: followed by its members.

Below are a few examples of list definitions:

```
lists = '''
## Lists used in the next examples
The following lists are only examples

First a list of banks and of the home country for each bank
>list BANKS = BANKS      : Danske  Nordea Ibs /
>                bankcountry : denmark sweden denmark /
>                include    :    0        1        1

A list of countries
>list country = country : denmark sweden

A list of different portfolios and of the regularity treatment
>list ports = ports      : households  NFC    RE  sov /
>                reg      : AIRB  AIRB  STA  STA

...
'''
```

Meaning

- `list BANKS = ...` defines a top-level list of banks with an **sublist** `bankcountry` that specifies where each bank belongs.
- `list country = ...` creates a country list.
- `list ports = ...` groups portfolios (`households`, `NFC`, `RE`, `sov`) and creates **sublists** with regulatory categories (`AIRB`, `STA`).

So the list statement starts with the keyword `list` and a `list name =`. Sublists are separated by `/` and each starts with a **sublist name**:

It makes sense to give the first sublist the same name as the list name.

The benefits of lists:

- **Compactness** – groups can be declared once and reused throughout the model.
- **Flexibility** – adding or removing members requires only updating the list, not the equations.
- **Transparency** – clear structure makes it easy to see which entities and attributes the model covers.

By defining lists in this way, ModelFlow provides a structured layer between the economic logic and the generated equations, enabling users to manage complex multi-entity models efficiently and transparently.

2.6.4 Do ... Enddo Loops

The `do ... enddo` construct allows repetitive patterns of equations to be generated automatically. When used together with **lists**, it enables the modeler to create a complete set of equations for every member of a list without having to write each one manually.

A `do` block defines a **loop** over the members of a specified list (for example, all banks, sectors, or countries).

Within the block, the loop variable can be referenced directly inside formulas — ModelFlow will automatically substitute the appropriate member name during expansion.

This mechanism is essential for large-scale models where the same behavioral relationships apply to multiple entities.

It ensures **consistency**, reduces **repetition**, and makes model specifications more **maintainable** and **transparent**.

Syntax

The syntax of a `do` block is straightforward:

- The loop *starts* with: `do list name` $\underbrace{\text{sublist name = value}}_{\text{optional}}$
- It *ends* with: `enddo`

One or more Frml statements — and possibly nested `do ... enddo` blocks — are written inside the loop.

Makemodel automatically expands each Frml statement for every member of the specified list that satisfies the optional filter.

Example

If the Python variable `lists` already defines some model lists, it can be combined with an example model specification and passed to **Makemodel** for processing.

The `show` property can then be used to display the resulting DSL equations after expansion.

```
list_example = Makemodel(lists +
'''
## Loop over all banks
>do banks
Calculate profit in each bank
>   profit__{banks} = revenue__{banks} - expenses__{banks}
Terminate the loop
> enddo
''')
list_example.show

FRML <> PROFIT__DANSKE = REVENUE__DANSKE -EXPENSES__DANSKE $
FRML <> PROFIT__NORDEA = REVENUE__NORDEA -EXPENSES__NORDEA $
FRML <> PROFIT__IBS = REVENUE__IBS -EXPENSES__IBS $

.showmd Will render whole Markdown text

list_example.render
```

2.7 Lists used in the next examples

The following lists are only examples

First a list of banks and of the home country for each bank

```
>list BANKS = BANKS      : Danske  Nordea Ibs /
>           bankcountry : denmark sweden denmark /
>           include     :    0      1      1
```

A list of countries

```
>list country = country : denmark sweden
```

A list of different portfolios and of the regulatory treatment

```
>list ports = ports      : households  NFC      RE  sov /
>           reg          : AIRB  AIRB  STA  STA
```

2.8 Loop over all banks

```
| >do banks
    Calculate profit in each bank
| >     profit__{banks} = revenue__{banks} - expenses__{banks}
    Terminate the loop
| > enddo
```

2.8.1 Conditional Loops

The `do` statement can also include conditions that restrict the loop to members of a list matching specific attribute values. This is written using the syntax `do =`.

For example, suppose the list `BANKS` includes a sublist `bankcountry`, then the following block will only expand for banks located in Denmark:

```
| Makemodel(lists +
| '''
| >do banks bankcountry = denmark
| >     profit__{banks} = revenue__{banks} - expenses__{banks}
| > enddo
| ''').show

FRML <> PROFIT__DANSKE = REVENUE__DANSKE -EXPENSES__DANSKE $
FRML <> PROFIT__IBS = REVENUE__IBS -EXPENSES__IBS $
```

ModelFlow will expand this block only for banks whose `bankcountry` attribute equals `denmark`.

This conditional looping makes it possible to write targeted relationships for specific subsets of entities for instance, defining different equations for domestic and foreign banks, or applying alternative behavioral rules to different sectors or regions.

By combining lists, sublists, and conditional `do` loops, ModelFlow allows highly structured models to be specified compactly while preserving clarity and flexibility.

2.8.2 Doable Equations

While `do ... enddo` loops provide an explicit way to expand formulas across members of a list, the **doable** construct offers a more concise and automatic alternative.

A **doable equation** is a *high-level shorthand* that combines the functionality of loops and summations without requiring the user to write repetitive loop structures.

Instead of manually specifying `do` blocks, ModelFlow can **detect the relevant lists automatically** by inspecting the **first** variable names in the equation.

Variables that contain double underscores (`__`) indicate **list dimensions**.

For example, in `profit__{banks}`, ModelFlow recognizes that `profit` is a variable defined over the list `BANKS`.

Similarly, in `loss__{banks}__{ports}`, the two underscores identify two list dimensions — `BANKS` and `PORTS`.

Box 2 Doable summary

$$\begin{array}{c}
 \text{doable } \underbrace{\langle \text{tag} \rangle}_{\text{Tag}} \underbrace{[\text{list} = \text{sublist} = \text{value}, \dots]}_{\text{Optional Conditions}} \underbrace{\text{VARIABLE}}_{\text{LHS Expression}} \underbrace{\text{__}\{ \text{list1} \} \text{__}\{ \text{list2} \} \dots}_{\text{LHS Dimensions}} \\
 \hspace{15em} \text{LHS} \\
 = \underbrace{\text{Expression including sublists}}_{\text{RHS Expression}}
 \end{array} \tag{2.1}$$

When ModelFlow encounters a `doable` equation such as:

```
Makemodel(lists + '''
>doable profit_{banks} = revenue_{banks} - expenses_{banks}
''').show

FRML <> PROFIT__DANSKE = REVENUE_DANSKE -EXPENSES_DANSKE $
FRML <> PROFIT__NORDEA = REVENUE_NORDEA -EXPENSES_NORDEA $
FRML <> PROFIT__IBS = REVENUE_IBS -EXPENSES_IBS $
```

it automatically expands the formula across all members of the list `BANKS` — just as a `do BANKS ... enddo` loop would.

Thus, the `doable` keyword eliminates the need for explicit looping syntax, making the specification both shorter and easier to read.

Doable equations can also include conditions and tags in the same way as `do` loops. For example:

```
Makemodel(lists+'>doable [banks= bankcountry=denmark] profit_{banks}=revenue_{banks} -expenses_{banks}').show

FRML <> PROFIT__DANSKE = REVENUE_DANSKE -EXPENSES_DANSKE $
FRML <> PROFIT__IBS = REVENUE_IBS -EXPENSES_IBS $
```

Expands only for banks where the sublist `bankcountry` equals `denmark`.

```
Makemodel(lists+'>doable [banks=include] profit_{banks}=revenue_{banks} -expenses_{banks}').show

FRML <> PROFIT__NORDEA = REVENUE_NORDEA -EXPENSES_NORDEA $
FRML <> PROFIT__IBS = REVENUE_IBS -EXPENSES_IBS $
```

Expands only for banks where the sublist `include` has the value 1

This mechanism is particularly powerful when dealing with complex, multi-dimensional systems — such as stress-test models with many banks and portfolios — since the modeler can write compact, pattern-based equations that ModelFlow expands automatically by detecting all relevant list dimensions.

In short, doable equations serve as an elegant, high-level syntax that:

- Avoids the boilerplate of `do ... enddo` structures,
- Automatically determines which lists apply by analyzing variable dimensions, and
- Keeps model specifications concise, readable, and scalable.

2.9 Sum of variables

To sum over lists the `sum` statement in expressions can be used. it will generate a sum over the list and potential filtered by a condition

```
Makemodel(lists+'>loss_danske = sum(ports,loss_{danske}_{ports})').show

FRML <> LOSS_DANSKE =
(LOSS_DANSKE__HOUSEHOLDS+LOSS_DANSKE__NFC+LOSS_DANSKE__RE+LOSS_DANSKE__SOV) $
```

2.9.1 Nested sums

```
Makemodel(lists+'>loss_total = sum(banks,sum(ports,loss_{banks}_{ports}))').show

FRML <> LOSS_TOTAL = ((LOSS_DANSKE__HOUSEHOLDS+LOSS_DANSKE__NFC+LOSS_DANSKE__RE+LOSS_DANSKE__SOV)+(LOSS_NORDEA__HOUSEHOLDS+LOSS_NORDEA__NFC+LOSS_NORDEA__RE+LOSS_NORDEA__SOV)+(LOSS_IBS__HOUSEHOLDS+LOSS_IBS__NFC+LOSS_IBS__RE+LOSS_IBS__SOV)) $
```

2.9.2 Conditional sum

Here the total sum for all banks for portfolios with `reg=sta`

```
Makemodel(lists+ '>loss_total_sta = sum(banks,sum(ports
reg=STA,loss_{banks}_{ports}))').show ↪

FRML <> LOSS_TOTAL_STA = ((LOSS__DANSKE__RE+LOSS__DANSKE__SOV)+(LOSS__NORDEA__RE+LOSS__NORDEA__SOV)+(LOSS__IBS__RE+LOSS__IBS__SOV)) $ ↪

Makemodel(lists+ '>loss_total_sta = sum(banks include,sum(ports
reg=STA,loss_{banks}_{ports}))').show ↪

FRML <> LOSS_TOTAL_STA = ↪
((LOSS__NORDEA__RE+LOSS__NORDEA__SOV)+(LOSS__IBS__RE+LOSS__IBS__SOV)) $ ↪
```

2.10 Combining doable and sum

In `doable` to create a sum of all the calculated Conditional the tag `sum` can be used. This will generate a sum of all the calculated values in the statement. `<sum=text>` will give dimension name to the result. If no text is given it will be called `sum`.

```
Makemodel(lists+
'>doable <sum=total> profit_{banks}=revenue_{banks} -expenses_{banks}'
).show

FRML <SUM=TOTAL> PROFIT__DANSKE = REVENUE__DANSKE -EXPENSES__DANSKE $
FRML <SUM=TOTAL> PROFIT__NORDEA = REVENUE__NORDEA -EXPENSES__NORDEA $
FRML <SUM=TOTAL> PROFIT__IBS = REVENUE__IBS -EXPENSES__IBS $
FRML <SUM=TOTAL> PROFIT__TOTAL = (PROFIT__DANSKE+PROFIT__NORDEA+PROFIT__IBS) $
```

And with conditions it works to

```
exploded = Makemodel(lists+
'>doable <sum=total> [banks bankcountry=denmark] profit_{banks}=revenue_{banks}
-expenses_{banks}'
)
exploded.show ↪

FRML <SUM=TOTAL> PROFIT__DANSKE = REVENUE__DANSKE -EXPENSES__DANSKE $
FRML <SUM=TOTAL> PROFIT__NORDEA = REVENUE__NORDEA -EXPENSES__NORDEA $
FRML <SUM=TOTAL> PROFIT__IBS = REVENUE__IBS -EXPENSES__IBS $
FRML <SUM=TOTAL> PROFIT__TOTAL = (PROFIT__DANSKE+PROFIT__NORDEA+PROFIT__IBS) $
```

2.10.1 .draw — Displaying the Model's Causal Structure

The `.draw` method visualizes the **causal structure** of a model as defined by its equations. It provides an intuitive graph of dependencies between variables, helping to understand the logical flow and interconnections within the model.

Warning

Use this feature only for **small models** — for larger models, the resulting diagram can become cluttered.

```
exploded.draw
```

The `Makemodel.draw` property is a convenient wrapper around `Makemodel.mmodel.drawmodel()`. It first accesses the `modelclass.model` instance defined by the DSL, then generates a graphical representation of the entire model.

All methods available to the underlying model instance can be used — for example, the `.tracedep` method, which allows tracing the dependencies of a single variables within the model.

```
exploded.mmodel.PROFIT__DANSKE.tracepre()
```

2.11 Tags and Formula Decoration

As mentioned earlier, the user should focus primarily on defining the **economic relationships** themselves.

However, in some cases it can be useful to **decorate** formulas to enable specific **modifications or behaviors** during calculations.

This is achieved by **tagging** the formula.

If we start with this simple formula and no tags. The formula will just be translated to a simple DSL formula:

```
| Makemodel('>a = b').show
```

```
| FRML <> A = B $
```

2.11.1 Enable fixing the value of a variable

When the value of a variable is known for certain time periods, it can be fixed by tagging its formula with `exo`. This is especially useful in forecasting exercises, where newly published statistics are available and should override the model's computed values.

When a formula is tagged with `exo`, two additional variables are automatically created:

- `_X` — the fixed (exogenous) value to be used.
- `_D` — a dummy variable that equals 1 in the periods where the fixed value should replace the model-generated one.

```
| Makemodel('><exo> a = b').show
```

```
| FRML <EXO> A = (B)* (1 -A_D)+ A_X*A_D $
```

this means that:

- When `A_D = 0`, the model uses the endogenous calculation $A = B$.
- When `A_D = 1`, the model instead uses the fixed value $A = A_X$.

This mechanism allows the modeller to incorporate known or published values seamlessly, without modifying the model's structure.

The `ModelFlow` model instance also includes convenient helper methods for managing these exogenous values — for example, to set, update, or release fixed variables during forecasting or simulation exercises.

2.11.2 Include add factors

In some it can be useful to be able to adjust the formula in order to make an experiment or to incorporate factors outside the model. Sometime called **add factors** or **expert judgements** To make life easy this can be done by using the tag `add` also the tag `add_suffic=` can be used to change the suffix for the add factor from the default `_A`

```
| Makemodel('><add> a = b').show
```

```
| FRML <ADD> A = (B + A_A) $
```

```
| FRML <CALC_ADD_FACTOR> A_A = (A) - (B) $
```

2.11.3 Reverse Calculation of Add Factors

A special `Frml` tagged with `CALC_ADD_FACTOR` is automatically generated by `Makemodel`.

This formula is used by the `ModelFlow` model instance to build a specialized *calculation model* that computes the **add factors** — the residual adjustments needed to reconcile actual or fixed values with the model's baseline equations.

On its own, this feature is not particularly useful, but when **combined with the `exo` tag**, it becomes powerful.

Together, they allow the model to **retain its marginal (behavioral) properties** even when some variables have **fixed values**.

After the add factors have been computed, the modeller can **reset the corresponding `_D` dummies to 0**, effectively releasing the fixed variables.

The model can then be used for **experiments, policy simulations, or shock analyses**, while the previously calculated add factors ensure internal consistency and preserve the model's original fit to observed data.

```
Makemodel('><exo,add> a = b').show

FRML <EXO,ADD> A = (B + A_A)* (1 -A_D)+ A_X*A_D  $

FRML <CALC_ADD_FACTOR>  A_A = (A) - (B) $
```

2.11.4 Naming of Add Factors

If the modeller prefers a **different naming convention** for add factors, this can be achieved by including the tag **`add_suffix=`** in the formula specification.

For example:

```
Makemodel('><add,add_suffix=__J> a = b').show

FRML <ADD,ADD_SUFFIX=__J> A = (B + A__J)  $

FRML <CALC_ADD_FACTOR>  A__J = (A) - (B) $
```

generate add factors named with the suffix `_J` instead of the default `_A`.

2.11.5 Including Calculated Fitted Values

When **add factors** or **fixed values** are used, it may be useful to compute the value implied by the **original fitted formula** — that is, the model's unadjusted behavioral relationship.

This can be achieved by tagging the formula with **`FIT`**.

When the **`FIT`** tag is applied, an additional formula is automatically created.

This formula calculates a new variable with the suffix **`_FITTED`**, representing the value generated by the **original model equation** (before the inclusion of add factors or fixed values).

This is helpful for comparing the model's baseline predictions with adjusted or constrained outcomes, especially in forecasting and diagnostic analysis.

```
Makemodel('><exo,add,fit> a = b').show

FRML <EXO,ADD,FIT> A = (B + A_A)* (1 -A_D)+ A_X*A_D  $
FRML <FIT>  A_FITTED = B $
FRML <CALC_ADD_FACTOR>  A_A = (A) - (B) $

Makemodel(lists+
'>doable <sum=total,exo,fit,add>  profit_{banks}=revenue_{banks} -expenses_{banks}'
).show

FRML <SUM=TOTAL,EXO,FIT,ADD> PROFIT__DANSKE = (REVENUE__DANSKE -EXPENSES__DANSKE +  →
PROFIT__DANSKE_A)* (1 -PROFIT__DANSKE_D)+ PROFIT__DANSKE_X*PROFIT__DANSKE_D  $
FRML <SUM=TOTAL,EXO,FIT,ADD> PROFIT__NORDEA = (REVENUE__NORDEA -EXPENSES__NORDEA +  →
PROFIT__NORDEA_A)* (1 -PROFIT__NORDEA_D)+ PROFIT__NORDEA_X*PROFIT__NORDEA_D  $
FRML <SUM=TOTAL,EXO,FIT,ADD> PROFIT__IBS = (REVENUE__IBS -EXPENSES__IBS +  →
PROFIT__IBS_A)* (1 -PROFIT__IBS_D)+ PROFIT__IBS_X*PROFIT__IBS_D  $
FRML <SUM=TOTAL,EXO,FIT,ADD> PROFIT__TOTAL =  →
((PROFIT__DANSKE+PROFIT__NORDEA+PROFIT__IBS) + PROFIT__TOTAL_A)* (1 -PROFIT__TOTAL_D)+  →
PROFIT__TOTAL_X*PROFIT__TOTAL_D  $
```

```

FRML <FIT> PROFIT__DANSKE_FITTED = REVENUE__DANSKE -EXPENSES__DANSKE $
FRML <FIT> PROFIT__NORDEA_FITTED = REVENUE__NORDEA -EXPENSES__NORDEA $
FRML <FIT> PROFIT__IBS_FITTED = REVENUE__IBS -EXPENSES__IBS $
FRML <FIT> PROFIT__TOTAL_FITTED = (PROFIT__DANSKE+PROFIT__NORDEA+PROFIT__IBS) $
FRML <CALC_ADD_FACTOR> PROFIT__DANSKE_A = (PROFIT__DANSKE) - (REVENUE__DANSKE
-EXPENSES__DANSKE) $
FRML <CALC_ADD_FACTOR> PROFIT__NORDEA_A = (PROFIT__NORDEA) - (REVENUE__NORDEA
-EXPENSES__NORDEA) $
FRML <CALC_ADD_FACTOR> PROFIT__IBS_A = (PROFIT__IBS) - (REVENUE__IBS -EXPENSES__IBS) $
FRML <CALC_ADD_FACTOR> PROFIT__TOTAL_A = (PROFIT__TOTAL) -
((PROFIT__DANSKE+PROFIT__NORDEA+PROFIT__IBS)) $

```

2.12 Using replacements= or str.replace() to simplify multi-dimensional formulas

When a formula includes several dimensions (e.g., `banks`, `country`, `ports`), variable names can become long and repetitive.

To keep model definitions compact and readable, you can define formulas with **placeholders** (such as `__dim`) and then insert the actual dimension names automatically.

There are now **two ways** to do this:

1. Use Python's built-in `str.replace()` directly in the formula string.
2. Use the `replacements=` argument in `Makemodel`, which applies the same logic internally.

The `replacements` argument accepts either a **single (pattern, replacement) tuple** or a **list of such tuples** for multiple substitutions.

2.12.1 Why this helps

- **Cleaner input:** Focus on the formula's logic instead of long dimension names.
- **Consistency:** Reuse the same placeholder pattern across multiple equations.
- **Maintainability:** Change dimensional naming once in the `replacements` list instead of editing all formulas.
- **Convenience:** The `replacements=` argument avoids having to call `.replace()` manually.

2.12.2 Example

Below, the same model is defined in two equivalent ways — first using `.replace()`, and then using the `replacements=` argument.

```

# Example 1 -- Using str.replace() manually
Makemodel(lists +
    '>doable <> loss__dim = portefolio__dim * pd__dim * lgd__dim'
    .replace('__dim', '__{banks}__{country}__{ports}'))
).show

FRML <> LOSS__DANSKE__DENMARK__HOUSEHOLDS = PORTEFOLIO__DANSKE__DENMARK__HOUSEHOLDS*P
D__DANSKE__DENMARK__HOUSEHOLDS*LGD__DANSKE__DENMARK__HOUSEHOLDS $
FRML <> LOSS__DANSKE__DENMARK__NFC =
PORTEFOLIO__DANSKE__DENMARK__NFC*PD__DANSKE__DENMARK__NFC*LGD__DANSKE__DENMARK__NFC $
FRML <> LOSS__DANSKE__DENMARK__RE =
PORTEFOLIO__DANSKE__DENMARK__RE*PD__DANSKE__DENMARK__RE*LGD__DANSKE__DENMARK__RE $
FRML <> LOSS__DANSKE__DENMARK__SOV =
PORTEFOLIO__DANSKE__DENMARK__SOV*PD__DANSKE__DENMARK__SOV*LGD__DANSKE__DENMARK__SOV $
FRML <> LOSS__DANSKE__SWEDEN__HOUSEHOLDS = PORTEFOLIO__DANSKE__SWEDEN__HOUSEHOLDS*PD_
DANSKE__SWEDEN__HOUSEHOLDS*LGD__DANSKE__SWEDEN__HOUSEHOLDS $
FRML <> LOSS__DANSKE__SWEDEN__NFC =
PORTEFOLIO__DANSKE__SWEDEN__NFC*PD__DANSKE__SWEDEN__NFC*LGD__DANSKE__SWEDEN__NFC $

```

```

FRML <> LOSS__DANSKE__SWEDEN__RE =
PORTEFOLIO__DANSKE__SWEDEN__RE*PD__DANSKE__SWEDEN__RE*LGD__DANSKE__SWEDEN__RE $
FRML <> LOSS__DANSKE__SWEDEN__SOV =
PORTEFOLIO__DANSKE__SWEDEN__SOV*PD__DANSKE__SWEDEN__SOV*LGD__DANSKE__SWEDEN__SOV $
FRML <> LOSS__NORDEA__DENMARK__HOUSEHOLDS = PORTEFOLIO__NORDEA__DENMARK__HOUSEHOLDS*P
D__NORDEA__DENMARK__HOUSEHOLDS*LGD__NORDEA__DENMARK__HOUSEHOLDS $
FRML <> LOSS__NORDEA__DENMARK__NFC =
PORTEFOLIO__NORDEA__DENMARK__NFC*PD__NORDEA__DENMARK__NFC*LGD__NORDEA__DENMARK__NFC $
FRML <> LOSS__NORDEA__DENMARK__RE =
PORTEFOLIO__NORDEA__DENMARK__RE*PD__NORDEA__DENMARK__RE*LGD__NORDEA__DENMARK__RE $
FRML <> LOSS__NORDEA__DENMARK__SOV =
PORTEFOLIO__NORDEA__DENMARK__SOV*PD__NORDEA__DENMARK__SOV*LGD__NORDEA__DENMARK__SOV $
FRML <> LOSS__NORDEA__SWEDEN__HOUSEHOLDS = PORTEFOLIO__NORDEA__SWEDEN__HOUSEHOLDS*PD
_NORDEA__SWEDEN__HOUSEHOLDS*LGD__NORDEA__SWEDEN__HOUSEHOLDS $
FRML <> LOSS__NORDEA__SWEDEN__NFC =
PORTEFOLIO__NORDEA__SWEDEN__NFC*PD__NORDEA__SWEDEN__NFC*LGD__NORDEA__SWEDEN__NFC $
FRML <> LOSS__NORDEA__SWEDEN__RE =
PORTEFOLIO__NORDEA__SWEDEN__RE*PD__NORDEA__SWEDEN__RE*LGD__NORDEA__SWEDEN__RE $
FRML <> LOSS__NORDEA__SWEDEN__SOV =
PORTEFOLIO__NORDEA__SWEDEN__SOV*PD__NORDEA__SWEDEN__SOV*LGD__NORDEA__SWEDEN__SOV $
FRML <> LOSS__IBS__DENMARK__HOUSEHOLDS = PORTEFOLIO__IBS__DENMARK__HOUSEHOLDS*PD__IBS
__DENMARK__HOUSEHOLDS*LGD__IBS__DENMARK__HOUSEHOLDS $
FRML <> LOSS__IBS__DENMARK__NFC =
PORTEFOLIO__IBS__DENMARK__NFC*PD__IBS__DENMARK__NFC*LGD__IBS__DENMARK__NFC $
FRML <> LOSS__IBS__DENMARK__RE =
PORTEFOLIO__IBS__DENMARK__RE*PD__IBS__DENMARK__RE*LGD__IBS__DENMARK__RE $
FRML <> LOSS__IBS__DENMARK__SOV =
PORTEFOLIO__IBS__DENMARK__SOV*PD__IBS__DENMARK__SOV*LGD__IBS__DENMARK__SOV $
FRML <> LOSS__IBS__SWEDEN__HOUSEHOLDS = PORTEFOLIO__IBS__SWEDEN__HOUSEHOLDS*PD__IBS
SWEDEN__HOUSEHOLDS*LGD__IBS__SWEDEN__HOUSEHOLDS $
FRML <> LOSS__IBS__SWEDEN__NFC =
PORTEFOLIO__IBS__SWEDEN__NFC*PD__IBS__SWEDEN__NFC*LGD__IBS__SWEDEN__NFC $
FRML <> LOSS__IBS__SWEDEN__RE =
PORTEFOLIO__IBS__SWEDEN__RE*PD__IBS__SWEDEN__RE*LGD__IBS__SWEDEN__RE $
FRML <> LOSS__IBS__SWEDEN__SOV =
PORTEFOLIO__IBS__SWEDEN__SOV*PD__IBS__SWEDEN__SOV*LGD__IBS__SWEDEN__SOV $

```

```
repl=('__dim', '__{banks}__{country}__{ports}')
```

```

Makemodel(lists+
'>doable <> loss__dim = portefolio__dim * pd__dim*lgd__dim',
replacements=repl ).show

```

```

FRML <> LOSS__DANSKE__DENMARK__HOUSEHOLDS = PORTEFOLIO__DANSKE__DENMARK__HOUSEHOLDS*P
D__DANSKE__DENMARK__HOUSEHOLDS*LGD__DANSKE__DENMARK__HOUSEHOLDS $
FRML <> LOSS__DANSKE__DENMARK__NFC =
PORTEFOLIO__DANSKE__DENMARK__NFC*PD__DANSKE__DENMARK__NFC*LGD__DANSKE__DENMARK__NFC $
FRML <> LOSS__DANSKE__DENMARK__RE =
PORTEFOLIO__DANSKE__DENMARK__RE*PD__DANSKE__DENMARK__RE*LGD__DANSKE__DENMARK__RE $
FRML <> LOSS__DANSKE__DENMARK__SOV =
PORTEFOLIO__DANSKE__DENMARK__SOV*PD__DANSKE__DENMARK__SOV*LGD__DANSKE__DENMARK__SOV $
FRML <> LOSS__DANSKE__SWEDEN__HOUSEHOLDS = PORTEFOLIO__DANSKE__SWEDEN__HOUSEHOLDS*PD
_DANSKE__SWEDEN__HOUSEHOLDS*LGD__DANSKE__SWEDEN__HOUSEHOLDS $
FRML <> LOSS__DANSKE__SWEDEN__NFC =
PORTEFOLIO__DANSKE__SWEDEN__NFC*PD__DANSKE__SWEDEN__NFC*LGD__DANSKE__SWEDEN__NFC $
FRML <> LOSS__DANSKE__SWEDEN__RE =
PORTEFOLIO__DANSKE__SWEDEN__RE*PD__DANSKE__SWEDEN__RE*LGD__DANSKE__SWEDEN__RE $
FRML <> LOSS__DANSKE__SWEDEN__SOV =
PORTEFOLIO__DANSKE__SWEDEN__SOV*PD__DANSKE__SWEDEN__SOV*LGD__DANSKE__SWEDEN__SOV $
FRML <> LOSS__NORDEA__DENMARK__HOUSEHOLDS = PORTEFOLIO__NORDEA__DENMARK__HOUSEHOLDS*P
D__NORDEA__DENMARK__HOUSEHOLDS*LGD__NORDEA__DENMARK__HOUSEHOLDS $

```

```

FRML <> LOSS__NORDEA__DENMARK__NFC =                                     ↪
PORTEFOLIO__NORDEA__DENMARK__NFC*PD__NORDEA__DENMARK__NFC*LGD__NORDEA__DENMARK__NFC $
FRML <> LOSS__NORDEA__DENMARK__RE =                                       ↪
PORTEFOLIO__NORDEA__DENMARK__RE*PD__NORDEA__DENMARK__RE*LGD__NORDEA__DENMARK__RE $
FRML <> LOSS__NORDEA__DENMARK__SOV =                                       ↪
PORTEFOLIO__NORDEA__DENMARK__SOV*PD__NORDEA__DENMARK__SOV*LGD__NORDEA__DENMARK__SOV $
FRML <> LOSS__NORDEA__SWEDEN__HOUSEHOLDS = PORTEFOLIO__NORDEA__SWEDEN__HOUSEHOLDS*PD_ ↪
_NORDEA__SWEDEN__HOUSEHOLDS*LGD__NORDEA__SWEDEN__HOUSEHOLDS $
FRML <> LOSS__NORDEA__SWEDEN__NFC =                                       ↪
PORTEFOLIO__NORDEA__SWEDEN__NFC*PD__NORDEA__SWEDEN__NFC*LGD__NORDEA__SWEDEN__NFC $
FRML <> LOSS__NORDEA__SWEDEN__RE =                                         ↪
PORTEFOLIO__NORDEA__SWEDEN__RE*PD__NORDEA__SWEDEN__RE*LGD__NORDEA__SWEDEN__RE $
FRML <> LOSS__NORDEA__SWEDEN__SOV =                                       ↪
PORTEFOLIO__NORDEA__SWEDEN__SOV*PD__NORDEA__SWEDEN__SOV*LGD__NORDEA__SWEDEN__SOV $
FRML <> LOSS__IBS__DENMARK__HOUSEHOLDS = PORTEFOLIO__IBS__DENMARK__HOUSEHOLDS*PD__IBS_ ↪
__DENMARK__HOUSEHOLDS*LGD__IBS__DENMARK__HOUSEHOLDS $
FRML <> LOSS__IBS__DENMARK__NFC =                                         ↪
PORTEFOLIO__IBS__DENMARK__NFC*PD__IBS__DENMARK__NFC*LGD__IBS__DENMARK__NFC $
FRML <> LOSS__IBS__DENMARK__RE =                                           ↪
PORTEFOLIO__IBS__DENMARK__RE*PD__IBS__DENMARK__RE*LGD__IBS__DENMARK__RE $
FRML <> LOSS__IBS__DENMARK__SOV =                                         ↪
PORTEFOLIO__IBS__DENMARK__SOV*PD__IBS__DENMARK__SOV*LGD__IBS__DENMARK__SOV $
FRML <> LOSS__IBS__SWEDEN__HOUSEHOLDS = PORTEFOLIO__IBS__SWEDEN__HOUSEHOLDS*PD__IBS__ ↪
SWEDEN__HOUSEHOLDS*LGD__IBS__SWEDEN__HOUSEHOLDS $
FRML <> LOSS__IBS__SWEDEN__NFC =                                           ↪
PORTEFOLIO__IBS__SWEDEN__NFC*PD__IBS__SWEDEN__NFC*LGD__IBS__SWEDEN__NFC $
FRML <> LOSS__IBS__SWEDEN__RE =                                           ↪
PORTEFOLIO__IBS__SWEDEN__RE*PD__IBS__SWEDEN__RE*LGD__IBS__SWEDEN__RE $
FRML <> LOSS__IBS__SWEDEN__SOV =                                           ↪
PORTEFOLIO__IBS__SWEDEN__SOV*PD__IBS__SWEDEN__SOV*LGD__IBS__SWEDEN__SOV $

```

2.13 Using list_defs= to define lists in Makemodel

In many cases, it can be convenient to provide **list definitions** as an argument to `Makemodel` rather than embedding them directly into the model specification string.

This approach helps separate the **technical structure** (lists, dimensions, replacements) from the **economic logic** (the actual formulas), making your model files cleaner and easier to read.

2.13.1 When to use list_defs=

- When several submodels share the **same list definitions**.
- When you want to keep the **economic equations visible**.

2.13.2 How it works

You can pass a string containing the list definitions directly to `Makemodel` using the `list_defs=` argument.

`Makemodel` automatically prepends these lists to the model specification before parsing and normalization.

```

Makemodel(lists +'>doable profit_{banks} = revenue_{banks} - expenses_{banks}').show

```

```
FRML <> PROFIT__DANSKE = REVENUE_DANSKE -EXPENSES_DANSKE $
FRML <> PROFIT__NORDEA = REVENUE_NORDEA -EXPENSES_NORDEA $
FRML <> PROFIT__IBS = REVENUE_IBS -EXPENSES_IBS $
```

```
Makemodel('>doable profit_{banks} = revenue_{banks} -
expenses_{banks}',list_defs=lists).show
```

→

```
FRML <> PROFIT__DANSKE = REVENUE_DANSKE -EXPENSES_DANSKE $
FRML <> PROFIT__NORDEA = REVENUE_NORDEA -EXPENSES_NORDEA $
FRML <> PROFIT__IBS = REVENUE_IBS -EXPENSES_IBS $
```

2.14 Using functools.partial to create simplified Makemodel constructors

When building many model components that share the same configuration — such as common `list_defs`, `replacements`, or `funks` — you can simplify your workflow by creating a **pre-configured version** of `Makemodel` using `functools.partial`.

This avoids repeating the same arguments for every model block and makes your code cleaner and more consistent.

2.14.1 Why this helps

- **Less repetition:** No need to retype the same `list_defs` or `replacements` each time.
- **Consistency:** All model blocks use identical definitions and structural setup.
- **Convenience:** The resulting constructor behaves exactly like `Makemodel`, but with defaults already filled in.

Example:

2.15 Building modular models by combining smaller formula groups

Complex models often consist of multiple logical components — such as behavioral equations, accounting identities, or sector-specific submodels.

Instead of defining one large block of equations in a single string, it's often more efficient to **specify smaller groups of formulas** and then combine them into a complete model using the `+` operator.

2.15.1 Why this approach is beneficial

- **Modularity:** Each `Makemodel` object can represent a distinct part of the model — for example, a banking block, a household sector, or a price system.
 - **Clarity:** Smaller, self-contained equation groups are easier to read, debug, and test independently.
 - **Reusability:** You can reuse model fragments across projects or scenarios by simply combining them differently.
 - **Incremental development:** Start with a minimal subset of formulas, verify behavior, and then expand the model progressively.
-

2.15.2 How it works

The `+` operator merges multiple `Makemodel` objects into a single `Listmodels` container, which behaves like a unified model.

This aggregation does not recompute anything — it simply combines their normalized forms, making it fast and memory-efficient.

Below, we define two small model blocks:

Each block is defined as an independent `Makemodel`, and then we combine them using `+`.

This creates a single integrated model (`Listmodels`) that can be displayed or solved as one unit.

In larger models, it's good practice to define separate logical components (blocks) for different economic or financial relationships.

Here, we define three related parts of a simple very stylized credit risk model:

1. **PD update block** – describes how each bank's probability of default (PD) evolves.
2. **Expected Loss block** – calculates expected losses as a function of exposure, PD, and loss given default (LGD).
3. **Bank Expected Loss block** – calculates expected losses for each bank.

Each block is created as an independent `Makemodel` object.

We then combine them with the `+` operator to form a complete integrated model (`Listmodels`), which can be solved or analyzed as one structure.

```
from functools import partial
SMakemodel = partial(Makemodel, list_defs=lists, replacements=('__dim',
'__{banks}__{country}__{ports}'))

SMakemodel('>doable <> loss__dim = portefolio__dim * pd__dim*lgd__dim').show

FRML <> LOSS__DANSKE__DENMARK__HOUSEHOLDS = PORTEFOLIO__DANSKE__DENMARK__HOUSEHOLDS*P_ →
D__DANSKE__DENMARK__HOUSEHOLDS*LGD__DANSKE__DENMARK__HOUSEHOLDS $
FRML <> LOSS__DANSKE__DENMARK__NFC = →
PORTEFOLIO__DANSKE__DENMARK__NFC*PD__DANSKE__DENMARK__NFC*LGD__DANSKE__DENMARK__NFC $
FRML <> LOSS__DANSKE__DENMARK__RE = →
PORTEFOLIO__DANSKE__DENMARK__RE*PD__DANSKE__DENMARK__RE*LGD__DANSKE__DENMARK__RE $
FRML <> LOSS__DANSKE__DENMARK__SOV = →
PORTEFOLIO__DANSKE__DENMARK__SOV*PD__DANSKE__DENMARK__SOV*LGD__DANSKE__DENMARK__SOV $
FRML <> LOSS__DANSKE__SWEDEN__HOUSEHOLDS = PORTEFOLIO__DANSKE__SWEDEN__HOUSEHOLDS*PD_ →
D__DANSKE__SWEDEN__HOUSEHOLDS*LGD__DANSKE__SWEDEN__HOUSEHOLDS $
FRML <> LOSS__DANSKE__SWEDEN__NFC = →
PORTEFOLIO__DANSKE__SWEDEN__NFC*PD__DANSKE__SWEDEN__NFC*LGD__DANSKE__SWEDEN__NFC $
FRML <> LOSS__DANSKE__SWEDEN__RE = →
PORTEFOLIO__DANSKE__SWEDEN__RE*PD__DANSKE__SWEDEN__RE*LGD__DANSKE__SWEDEN__RE $
FRML <> LOSS__DANSKE__SWEDEN__SOV = →
PORTEFOLIO__DANSKE__SWEDEN__SOV*PD__DANSKE__SWEDEN__SOV*LGD__DANSKE__SWEDEN__SOV $
FRML <> LOSS__NORDEA__DENMARK__HOUSEHOLDS = PORTEFOLIO__NORDEA__DENMARK__HOUSEHOLDS*P_ →
D__NORDEA__DENMARK__HOUSEHOLDS*LGD__NORDEA__DENMARK__HOUSEHOLDS $
FRML <> LOSS__NORDEA__DENMARK__NFC = →
PORTEFOLIO__NORDEA__DENMARK__NFC*PD__NORDEA__DENMARK__NFC*LGD__NORDEA__DENMARK__NFC $
FRML <> LOSS__NORDEA__DENMARK__RE = →
PORTEFOLIO__NORDEA__DENMARK__RE*PD__NORDEA__DENMARK__RE*LGD__NORDEA__DENMARK__RE $
FRML <> LOSS__NORDEA__DENMARK__SOV = →
PORTEFOLIO__NORDEA__DENMARK__SOV*PD__NORDEA__DENMARK__SOV*LGD__NORDEA__DENMARK__SOV $
FRML <> LOSS__NORDEA__SWEDEN__HOUSEHOLDS = PORTEFOLIO__NORDEA__SWEDEN__HOUSEHOLDS*PD_ →
D__NORDEA__SWEDEN__HOUSEHOLDS*LGD__NORDEA__SWEDEN__HOUSEHOLDS $
FRML <> LOSS__NORDEA__SWEDEN__NFC = →
PORTEFOLIO__NORDEA__SWEDEN__NFC*PD__NORDEA__SWEDEN__NFC*LGD__NORDEA__SWEDEN__NFC $
FRML <> LOSS__NORDEA__SWEDEN__RE = →
PORTEFOLIO__NORDEA__SWEDEN__RE*PD__NORDEA__SWEDEN__RE*LGD__NORDEA__SWEDEN__RE $
```

```

FRML <> LOSS__NORDEA__SWEDEN__SOV =
PORTEFOLIO__NORDEA__SWEDEN__SOV*PD__NORDEA__SWEDEN__SOV*LGD__NORDEA__SWEDEN__SOV $
FRML <> LOSS__IBS__DENMARK__HOUSEHOLDS = PORTEFOLIO__IBS__DENMARK__HOUSEHOLDS*PD__IBS__
__DENMARK__HOUSEHOLDS*LGD__IBS__DENMARK__HOUSEHOLDS $
FRML <> LOSS__IBS__DENMARK__NFC =
PORTEFOLIO__IBS__DENMARK__NFC*PD__IBS__DENMARK__NFC*LGD__IBS__DENMARK__NFC $
FRML <> LOSS__IBS__DENMARK__RE =
PORTEFOLIO__IBS__DENMARK__RE*PD__IBS__DENMARK__RE*LGD__IBS__DENMARK__RE $
FRML <> LOSS__IBS__DENMARK__SOV =
PORTEFOLIO__IBS__DENMARK__SOV*PD__IBS__DENMARK__SOV*LGD__IBS__DENMARK__SOV $
FRML <> LOSS__IBS__SWEDEN__HOUSEHOLDS = PORTEFOLIO__IBS__SWEDEN__HOUSEHOLDS*PD__IBS__
SWEDEN__HOUSEHOLDS*LGD__IBS__SWEDEN__HOUSEHOLDS $
FRML <> LOSS__IBS__SWEDEN__NFC =
PORTEFOLIO__IBS__SWEDEN__NFC*PD__IBS__SWEDEN__NFC*LGD__IBS__SWEDEN__NFC $
FRML <> LOSS__IBS__SWEDEN__RE =
PORTEFOLIO__IBS__SWEDEN__RE*PD__IBS__SWEDEN__RE*LGD__IBS__SWEDEN__RE $
FRML <> LOSS__IBS__SWEDEN__SOV =
PORTEFOLIO__IBS__SWEDEN__SOV*PD__IBS__SWEDEN__SOV*LGD__IBS__SWEDEN__SOV $

# - - - Define three small, modular formula groups - - -
# first define the replacements
repl = ('__dim', '__{banks}__{country}__{ports}__')
# - - - Block 1: PD update equations - - -
pd_update_block = Makemodel(lists+
'>doable <> pd__dim = 0.02 + 0.8 * pd__dim(-1) + 0.1 *
gdp_growth__{country}',replacements=repl)

# - - - Block 2: Expected loss calculation - - -
expected_loss_block = Makemodel(lists +
'>doable <> el__dim = exposure__dim * pd__dim * lgd__dim',replacements=repl)

# - - - Block 3: Expected loss calculation per bank - - -
bank_block = Makemodel(lists +
'''
> do banks
>     Expected_loss_{banks} = sum(country, sum(ports, el__dim))
> enddo
'''
,replacements=repl)

# - - - Combine all blocks into one model - - -
credit_risk_model = sum([pd_update_block + expected_loss_block + bank_block])
credit_risk_model.show

FRML <> PD__DANSKE__DENMARK__HOUSEHOLDS = 0.02+0.8*PD__DANSKE__DENMARK__HOUSEHOLDS(
-1)+0.1*GDP_GROWTH__DENMARK $
FRML <> PD__DANSKE__DENMARK__NFC = 0.02+0.8*PD__DANSKE__DENMARK__NFC(
-1)+0.1*GDP_GROWTH__DENMARK $
FRML <> PD__DANSKE__DENMARK__RE = 0.02+0.8*PD__DANSKE__DENMARK__RE(
-1)+0.1*GDP_GROWTH__DENMARK $
FRML <> PD__DANSKE__DENMARK__SOV = 0.02+0.8*PD__DANSKE__DENMARK__SOV(
-1)+0.1*GDP_GROWTH__DENMARK $
FRML <> PD__DANSKE__SWEDEN__HOUSEHOLDS = 0.02+0.8*PD__DANSKE__SWEDEN__HOUSEHOLDS(
-1)+0.1*GDP_GROWTH__SWEDEN $
FRML <> PD__DANSKE__SWEDEN__NFC = 0.02+0.8*PD__DANSKE__SWEDEN__NFC(
-1)+0.1*GDP_GROWTH__SWEDEN $
FRML <> PD__DANSKE__SWEDEN__RE = 0.02+0.8*PD__DANSKE__SWEDEN__RE(
-1)+0.1*GDP_GROWTH__SWEDEN $
FRML <> PD__DANSKE__SWEDEN__SOV = 0.02+0.8*PD__DANSKE__SWEDEN__SOV(
-1)+0.1*GDP_GROWTH__SWEDEN $
FRML <> PD__NORDEA__DENMARK__HOUSEHOLDS = 0.02+0.8*PD__NORDEA__DENMARK__HOUSEHOLDS(
-1)+0.1*GDP_GROWTH__DENMARK $

```

```

FRML <> PD__NORDEA__DENMARK__NFC = 0.02+0.8*PD__NORDEA__DENMARK__NFC(
-1)+0.1*GDP_GROWTH__DENMARK $
FRML <> PD__NORDEA__DENMARK__RE = 0.02+0.8*PD__NORDEA__DENMARK__RE(
-1)+0.1*GDP_GROWTH__DENMARK $
FRML <> PD__NORDEA__DENMARK__SOV = 0.02+0.8*PD__NORDEA__DENMARK__SOV(
-1)+0.1*GDP_GROWTH__DENMARK $
FRML <> PD__NORDEA__SWEDEN__HOUSEHOLDS = 0.02+0.8*PD__NORDEA__SWEDEN__HOUSEHOLDS(
-1)+0.1*GDP_GROWTH__SWEDEN $
FRML <> PD__NORDEA__SWEDEN__NFC = 0.02+0.8*PD__NORDEA__SWEDEN__NFC(
-1)+0.1*GDP_GROWTH__SWEDEN $
FRML <> PD__NORDEA__SWEDEN__RE = 0.02+0.8*PD__NORDEA__SWEDEN__RE(
-1)+0.1*GDP_GROWTH__SWEDEN $
FRML <> PD__NORDEA__SWEDEN__SOV = 0.02+0.8*PD__NORDEA__SWEDEN__SOV(
-1)+0.1*GDP_GROWTH__SWEDEN $
FRML <> PD__IBS__DENMARK__HOUSEHOLDS = 0.02+0.8*PD__IBS__DENMARK__HOUSEHOLDS(
-1)+0.1*GDP_GROWTH__DENMARK $
FRML <> PD__IBS__DENMARK__NFC = 0.02+0.8*PD__IBS__DENMARK__NFC(
-1)+0.1*GDP_GROWTH__DENMARK $
FRML <> PD__IBS__DENMARK__RE = 0.02+0.8*PD__IBS__DENMARK__RE(
-1)+0.1*GDP_GROWTH__DENMARK $
FRML <> PD__IBS__DENMARK__SOV = 0.02+0.8*PD__IBS__DENMARK__SOV(
-1)+0.1*GDP_GROWTH__DENMARK $
FRML <> PD__IBS__SWEDEN__HOUSEHOLDS = 0.02+0.8*PD__IBS__SWEDEN__HOUSEHOLDS(
-1)+0.1*GDP_GROWTH__SWEDEN $
FRML <> PD__IBS__SWEDEN__NFC = 0.02+0.8*PD__IBS__SWEDEN__NFC( -1)+0.1*GDP_GROWTH__SWEDEN $
FRML <> PD__IBS__SWEDEN__RE = 0.02+0.8*PD__IBS__SWEDEN__RE( -1)+0.1*GDP_GROWTH__SWEDEN $
FRML <> PD__IBS__SWEDEN__SOV = 0.02+0.8*PD__IBS__SWEDEN__SOV( -1)+0.1*GDP_GROWTH__SWEDEN $
FRML <> EL__DANSKE__DENMARK__HOUSEHOLDS = EXPOSURE__DANSKE__DENMARK__HOUSEHOLDS*PD__D
ANSKE__DENMARK__HOUSEHOLDS*LGD__DANSKE__DENMARK__HOUSEHOLDS $
FRML <> EL__DANSKE__DENMARK__NFC =
EXPOSURE__DANSKE__DENMARK__NFC*PD__DANSKE__DENMARK__NFC*LGD__DANSKE__DENMARK__NFC $
FRML <> EL__DANSKE__DENMARK__RE =
EXPOSURE__DANSKE__DENMARK__RE*PD__DANSKE__DENMARK__RE*LGD__DANSKE__DENMARK__RE $
FRML <> EL__DANSKE__DENMARK__SOV =
EXPOSURE__DANSKE__DENMARK__SOV*PD__DANSKE__DENMARK__SOV*LGD__DANSKE__DENMARK__SOV $
FRML <> EL__DANSKE__SWEDEN__HOUSEHOLDS = EXPOSURE__DANSKE__SWEDEN__HOUSEHOLDS*PD__DAN
SKE__SWEDEN__HOUSEHOLDS*LGD__DANSKE__SWEDEN__HOUSEHOLDS $
FRML <> EL__DANSKE__SWEDEN__NFC =
EXPOSURE__DANSKE__SWEDEN__NFC*PD__DANSKE__SWEDEN__NFC*LGD__DANSKE__SWEDEN__NFC $
FRML <> EL__DANSKE__SWEDEN__RE =
EXPOSURE__DANSKE__SWEDEN__RE*PD__DANSKE__SWEDEN__RE*LGD__DANSKE__SWEDEN__RE $
FRML <> EL__DANSKE__SWEDEN__SOV =
EXPOSURE__DANSKE__SWEDEN__SOV*PD__DANSKE__SWEDEN__SOV*LGD__DANSKE__SWEDEN__SOV $
FRML <> EL__NORDEA__DENMARK__HOUSEHOLDS = EXPOSURE__NORDEA__DENMARK__HOUSEHOLDS*PD__N
ORDEA__DENMARK__HOUSEHOLDS*LGD__NORDEA__DENMARK__HOUSEHOLDS $
FRML <> EL__NORDEA__DENMARK__NFC =
EXPOSURE__NORDEA__DENMARK__NFC*PD__NORDEA__DENMARK__NFC*LGD__NORDEA__DENMARK__NFC $
FRML <> EL__NORDEA__DENMARK__RE =
EXPOSURE__NORDEA__DENMARK__RE*PD__NORDEA__DENMARK__RE*LGD__NORDEA__DENMARK__RE $
FRML <> EL__NORDEA__DENMARK__SOV =
EXPOSURE__NORDEA__DENMARK__SOV*PD__NORDEA__DENMARK__SOV*LGD__NORDEA__DENMARK__SOV $
FRML <> EL__NORDEA__SWEDEN__HOUSEHOLDS = EXPOSURE__NORDEA__SWEDEN__HOUSEHOLDS*PD__NOR
DEA__SWEDEN__HOUSEHOLDS*LGD__NORDEA__SWEDEN__HOUSEHOLDS $
FRML <> EL__NORDEA__SWEDEN__NFC =
EXPOSURE__NORDEA__SWEDEN__NFC*PD__NORDEA__SWEDEN__NFC*LGD__NORDEA__SWEDEN__NFC $
FRML <> EL__NORDEA__SWEDEN__RE =
EXPOSURE__NORDEA__SWEDEN__RE*PD__NORDEA__SWEDEN__RE*LGD__NORDEA__SWEDEN__RE $
FRML <> EL__NORDEA__SWEDEN__SOV =
EXPOSURE__NORDEA__SWEDEN__SOV*PD__NORDEA__SWEDEN__SOV*LGD__NORDEA__SWEDEN__SOV $
FRML <> EL__IBS__DENMARK__HOUSEHOLDS = EXPOSURE__IBS__DENMARK__HOUSEHOLDS*PD__IBS__DE
NMARK__HOUSEHOLDS*LGD__IBS__DENMARK__HOUSEHOLDS $

```

```

FRML <> EL__IBS__DENMARK__NFC =
EXPOSURE__IBS__DENMARK__NFC*PD__IBS__DENMARK__NFC*LGD__IBS__DENMARK__NFC $
FRML <> EL__IBS__DENMARK__RE =
EXPOSURE__IBS__DENMARK__RE*PD__IBS__DENMARK__RE*LGD__IBS__DENMARK__RE $
FRML <> EL__IBS__DENMARK__SOV =
EXPOSURE__IBS__DENMARK__SOV*PD__IBS__DENMARK__SOV*LGD__IBS__DENMARK__SOV $
FRML <> EL__IBS__SWEDEN__HOUSEHOLDS = EXPOSURE__IBS__SWEDEN__HOUSEHOLDS*PD__IBS__SWEDEN__HOUSEHOLDS*LGD__IBS__SWEDEN__HOUSEHOLDS $
FRML <> EL__IBS__SWEDEN__NFC =
EXPOSURE__IBS__SWEDEN__NFC*PD__IBS__SWEDEN__NFC*LGD__IBS__SWEDEN__NFC $
FRML <> EL__IBS__SWEDEN__RE =
EXPOSURE__IBS__SWEDEN__RE*PD__IBS__SWEDEN__RE*LGD__IBS__SWEDEN__RE $
FRML <> EL__IBS__SWEDEN__SOV =
EXPOSURE__IBS__SWEDEN__SOV*PD__IBS__SWEDEN__SOV*LGD__IBS__SWEDEN__SOV $
FRML <> EXPECTED_LOSS_DANSKE = ((EL__DANSKE__DENMARK__HOUSEHOLDS+EL__DANSKE__DENMARK__NFC+EL__DANSKE__DENMARK__RE+EL__DANSKE__DENMARK__SOV)+(EL__DANSKE__SWEDEN__HOUSEHOLDS+EL__DANSKE__SWEDEN__NFC+EL__DANSKE__SWEDEN__RE+EL__DANSKE__SWEDEN__SOV)) $
FRML <> EXPECTED_LOSS_NORDEA = ((EL__NORDEA__DENMARK__HOUSEHOLDS+EL__NORDEA__DENMARK__NFC+EL__NORDEA__DENMARK__RE+EL__NORDEA__DENMARK__SOV)+(EL__NORDEA__SWEDEN__HOUSEHOLDS+EL__NORDEA__SWEDEN__NFC+EL__NORDEA__SWEDEN__RE+EL__NORDEA__SWEDEN__SOV)) $
FRML <> EXPECTED_LOSS_IBS = ((EL__IBS__DENMARK__HOUSEHOLDS+EL__IBS__DENMARK__NFC+EL__IBS__DENMARK__RE+EL__IBS__DENMARK__SOV)+(EL__IBS__SWEDEN__HOUSEHOLDS+EL__IBS__SWEDEN__NFC+EL__IBS__SWEDEN__RE+EL__IBS__SWEDEN__SOV)) $

```

Of course we could also just make the model in one Makemodel or even simpler a SMakemodel that was defined above.

```

total_credit_risk_model = SMakemodel(''')
>doable <> pd__dim = 0.02 + 0.8 * pd__dim(-1) + 0.1 * gdp_growth_{country}
>doable <> el__dim = exposure__dim * pd__dim * lgd__dim
>do banks
> Expected_loss_{banks} = sum(country, sum(ports, el__dim))
>enddo ''')
total_credit_risk_model.show

```

```

FRML <> PD__DANSKE__DENMARK__HOUSEHOLDS = 0.02+0.8*PD__DANSKE__DENMARK__HOUSEHOLDS(-1)+0.1*GDP_GROWTH__DENMARK $
FRML <> PD__DANSKE__DENMARK__NFC = 0.02+0.8*PD__DANSKE__DENMARK__NFC(-1)+0.1*GDP_GROWTH__DENMARK $
FRML <> PD__DANSKE__DENMARK__RE = 0.02+0.8*PD__DANSKE__DENMARK__RE(-1)+0.1*GDP_GROWTH__DENMARK $
FRML <> PD__DANSKE__DENMARK__SOV = 0.02+0.8*PD__DANSKE__DENMARK__SOV(-1)+0.1*GDP_GROWTH__DENMARK $
FRML <> PD__DANSKE__SWEDEN__HOUSEHOLDS = 0.02+0.8*PD__DANSKE__SWEDEN__HOUSEHOLDS(-1)+0.1*GDP_GROWTH__SWEDEN $
FRML <> PD__DANSKE__SWEDEN__NFC = 0.02+0.8*PD__DANSKE__SWEDEN__NFC(-1)+0.1*GDP_GROWTH__SWEDEN $
FRML <> PD__DANSKE__SWEDEN__RE = 0.02+0.8*PD__DANSKE__SWEDEN__RE(-1)+0.1*GDP_GROWTH__SWEDEN $
FRML <> PD__DANSKE__SWEDEN__SOV = 0.02+0.8*PD__DANSKE__SWEDEN__SOV(-1)+0.1*GDP_GROWTH__SWEDEN $
FRML <> PD__NORDEA__DENMARK__HOUSEHOLDS = 0.02+0.8*PD__NORDEA__DENMARK__HOUSEHOLDS(-1)+0.1*GDP_GROWTH__DENMARK $
FRML <> PD__NORDEA__DENMARK__NFC = 0.02+0.8*PD__NORDEA__DENMARK__NFC(-1)+0.1*GDP_GROWTH__DENMARK $
FRML <> PD__NORDEA__DENMARK__RE = 0.02+0.8*PD__NORDEA__DENMARK__RE(-1)+0.1*GDP_GROWTH__DENMARK $
FRML <> PD__NORDEA__DENMARK__SOV = 0.02+0.8*PD__NORDEA__DENMARK__SOV(-1)+0.1*GDP_GROWTH__DENMARK $

```

```

FRML <> PD__NORDEA__SWEDEN__HOUSEHOLDS = 0.02+0.8*PD__NORDEA__SWEDEN__HOUSEHOLDS( →
-1)+0.1*GDP_GROWTH__SWEDEN $
FRML <> PD__NORDEA__SWEDEN__NFC = 0.02+0.8*PD__NORDEA__SWEDEN__NFC( →
-1)+0.1*GDP_GROWTH__SWEDEN $
FRML <> PD__NORDEA__SWEDEN__RE = 0.02+0.8*PD__NORDEA__SWEDEN__RE( →
-1)+0.1*GDP_GROWTH__SWEDEN $
FRML <> PD__NORDEA__SWEDEN__SOV = 0.02+0.8*PD__NORDEA__SWEDEN__SOV( →
-1)+0.1*GDP_GROWTH__SWEDEN $
FRML <> PD__IBS__DENMARK__HOUSEHOLDS = 0.02+0.8*PD__IBS__DENMARK__HOUSEHOLDS( →
-1)+0.1*GDP_GROWTH__DENMARK $
FRML <> PD__IBS__DENMARK__NFC = 0.02+0.8*PD__IBS__DENMARK__NFC( →
-1)+0.1*GDP_GROWTH__DENMARK $
FRML <> PD__IBS__DENMARK__RE = 0.02+0.8*PD__IBS__DENMARK__RE( →
-1)+0.1*GDP_GROWTH__DENMARK $
FRML <> PD__IBS__DENMARK__SOV = 0.02+0.8*PD__IBS__DENMARK__SOV( →
-1)+0.1*GDP_GROWTH__DENMARK $
FRML <> PD__IBS__SWEDEN__HOUSEHOLDS = 0.02+0.8*PD__IBS__SWEDEN__HOUSEHOLDS( →
-1)+0.1*GDP_GROWTH__SWEDEN $
FRML <> PD__IBS__SWEDEN__NFC = 0.02+0.8*PD__IBS__SWEDEN__NFC( -1)+0.1*GDP_GROWTH__SWEDEN $
FRML <> PD__IBS__SWEDEN__RE = 0.02+0.8*PD__IBS__SWEDEN__RE( -1)+0.1*GDP_GROWTH__SWEDEN $
FRML <> PD__IBS__SWEDEN__SOV = 0.02+0.8*PD__IBS__SWEDEN__SOV( -1)+0.1*GDP_GROWTH__SWEDEN $
FRML <> EL__DANSKE__DENMARK__HOUSEHOLDS = EXPOSURE__DANSKE__DENMARK__HOUSEHOLDS*PD__D →
ANSKE__DENMARK__HOUSEHOLDS*LGD__DANSKE__DENMARK__HOUSEHOLDS $
FRML <> EL__DANSKE__DENMARK__NFC = →
EXPOSURE__DANSKE__DENMARK__NFC*PD__DANSKE__DENMARK__NFC*LGD__DANSKE__DENMARK__NFC $
FRML <> EL__DANSKE__DENMARK__RE = →
EXPOSURE__DANSKE__DENMARK__RE*PD__DANSKE__DENMARK__RE*LGD__DANSKE__DENMARK__RE $
FRML <> EL__DANSKE__DENMARK__SOV = →
EXPOSURE__DANSKE__DENMARK__SOV*PD__DANSKE__DENMARK__SOV*LGD__DANSKE__DENMARK__SOV $
FRML <> EL__DANSKE__SWEDEN__HOUSEHOLDS = EXPOSURE__DANSKE__SWEDEN__HOUSEHOLDS*PD__DAN →
SKE__SWEDEN__HOUSEHOLDS*LGD__DANSKE__SWEDEN__HOUSEHOLDS $
FRML <> EL__DANSKE__SWEDEN__NFC = →
EXPOSURE__DANSKE__SWEDEN__NFC*PD__DANSKE__SWEDEN__NFC*LGD__DANSKE__SWEDEN__NFC $
FRML <> EL__DANSKE__SWEDEN__RE = →
EXPOSURE__DANSKE__SWEDEN__RE*PD__DANSKE__SWEDEN__RE*LGD__DANSKE__SWEDEN__RE $
FRML <> EL__DANSKE__SWEDEN__SOV = →
EXPOSURE__DANSKE__SWEDEN__SOV*PD__DANSKE__SWEDEN__SOV*LGD__DANSKE__SWEDEN__SOV $
FRML <> EL__NORDEA__DENMARK__HOUSEHOLDS = EXPOSURE__NORDEA__DENMARK__HOUSEHOLDS*PD__N →
ORDEA__DENMARK__HOUSEHOLDS*LGD__NORDEA__DENMARK__HOUSEHOLDS $
FRML <> EL__NORDEA__DENMARK__NFC = →
EXPOSURE__NORDEA__DENMARK__NFC*PD__NORDEA__DENMARK__NFC*LGD__NORDEA__DENMARK__NFC $
FRML <> EL__NORDEA__DENMARK__RE = →
EXPOSURE__NORDEA__DENMARK__RE*PD__NORDEA__DENMARK__RE*LGD__NORDEA__DENMARK__RE $
FRML <> EL__NORDEA__DENMARK__SOV = →
EXPOSURE__NORDEA__DENMARK__SOV*PD__NORDEA__DENMARK__SOV*LGD__NORDEA__DENMARK__SOV $
FRML <> EL__NORDEA__SWEDEN__HOUSEHOLDS = EXPOSURE__NORDEA__SWEDEN__HOUSEHOLDS*PD__NOR →
DEA__SWEDEN__HOUSEHOLDS*LGD__NORDEA__SWEDEN__HOUSEHOLDS $
FRML <> EL__NORDEA__SWEDEN__NFC = →
EXPOSURE__NORDEA__SWEDEN__NFC*PD__NORDEA__SWEDEN__NFC*LGD__NORDEA__SWEDEN__NFC $
FRML <> EL__NORDEA__SWEDEN__RE = →
EXPOSURE__NORDEA__SWEDEN__RE*PD__NORDEA__SWEDEN__RE*LGD__NORDEA__SWEDEN__RE $
FRML <> EL__NORDEA__SWEDEN__SOV = →
EXPOSURE__NORDEA__SWEDEN__SOV*PD__NORDEA__SWEDEN__SOV*LGD__NORDEA__SWEDEN__SOV $
FRML <> EL__IBS__DENMARK__HOUSEHOLDS = EXPOSURE__IBS__DENMARK__HOUSEHOLDS*PD__IBS__DE →
NMARK__HOUSEHOLDS*LGD__IBS__DENMARK__HOUSEHOLDS $
FRML <> EL__IBS__DENMARK__NFC = →
EXPOSURE__IBS__DENMARK__NFC*PD__IBS__DENMARK__NFC*LGD__IBS__DENMARK__NFC $
FRML <> EL__IBS__DENMARK__RE = →
EXPOSURE__IBS__DENMARK__RE*PD__IBS__DENMARK__RE*LGD__IBS__DENMARK__RE $
FRML <> EL__IBS__DENMARK__SOV = →
EXPOSURE__IBS__DENMARK__SOV*PD__IBS__DENMARK__SOV*LGD__IBS__DENMARK__SOV $

```

```

FRML <> EL__IBS__SWEDEN__HOUSEHOLDS = EXPOSURE__IBS__SWEDEN__HOUSEHOLDS*PD__IBS__SWEDEN__HOUSEHOLDS*LGD__IBS__SWEDEN__HOUSEHOLDS $
FRML <> EL__IBS__SWEDEN__NFC =
EXPOSURE__IBS__SWEDEN__NFC*PD__IBS__SWEDEN__NFC*LGD__IBS__SWEDEN__NFC $
FRML <> EL__IBS__SWEDEN__RE =
EXPOSURE__IBS__SWEDEN__RE*PD__IBS__SWEDEN__RE*LGD__IBS__SWEDEN__RE $
FRML <> EL__IBS__SWEDEN__SOV =
EXPOSURE__IBS__SWEDEN__SOV*PD__IBS__SWEDEN__SOV*LGD__IBS__SWEDEN__SOV $
FRML <> EXPECTED_LOSS_DANSKE = ((EL__DANSKE__DENMARK__HOUSEHOLDS+EL__DANSKE__DENMARK__NFC+EL__DANSKE__DENMARK__RE+EL__DANSKE__DENMARK__SOV)+(EL__DANSKE__SWEDEN__HOUSEHOLDS+EL__DANSKE__SWEDEN__NFC+EL__DANSKE__SWEDEN__RE+EL__DANSKE__SWEDEN__SOV)) $
FRML <> EXPECTED_LOSS_NORDEA = ((EL__NORDEA__DENMARK__HOUSEHOLDS+EL__NORDEA__DENMARK__NFC+EL__NORDEA__DENMARK__RE+EL__NORDEA__DENMARK__SOV)+(EL__NORDEA__SWEDEN__HOUSEHOLDS+EL__NORDEA__SWEDEN__NFC+EL__NORDEA__SWEDEN__RE+EL__NORDEA__SWEDEN__SOV)) $
FRML <> EXPECTED_LOSS_IBS = ((EL__IBS__DENMARK__HOUSEHOLDS+EL__IBS__DENMARK__NFC+EL__IBS__DENMARK__RE+EL__IBS__DENMARK__SOV)+(EL__IBS__SWEDEN__HOUSEHOLDS+EL__IBS__SWEDEN__NFC+EL__IBS__SWEDEN__RE+EL__IBS__SWEDEN__SOV)) $

```

And the causal structure can be displayed

```
credit_risk_model.draw
```

2.16 Estimating behavioural equations

So far every coefficient has been written directly into the formulas, and each formula holds exactly — it is an identity. A macro-structural model also contains **behavioural equations** whose coefficients are **estimated** from data. `Makemodel` can run the estimation *while the model is built*: an equation tagged with `<estimator=...>` is estimated, and the estimated coefficients are baked into the expression before it is normalized into a FRML.

This section covers the default estimation backend `Estimate_nls` — non-linear least squares based on the `lmfit` package. (`Estimate_nls` is a factory class; with the default `solver='lmfit'` it returns an `Estimate_nls_lmfit` instance.)

2.16.1 Coefficient placeholders

Coefficients to be estimated are written as `C(1)`, `C(2)`, ... in the equation. With the `param_names=` option meaningful names can be used instead: `param_names=['lambda', 'beta_longterm', 'beta_shortterm']` allows placeholders like `LAMBDA`, `BETA_LONGTERM(1)` and `BETA_SHORTTERM(10)`, making for instance an ECM equation self-documenting. A bare name like `LAMBDA` is shorthand for placeholder number 1.

2.16.2 A default estimator

Boiler-plate options are collected once with the `.with_defaults()` factory. The resulting callable — here named `ls` — is then referenced by name in equation tags:

```

import numpy as np
import pandas as pd
from modelestimators import Estimate_nls

# A small synthetic dataset: CON is (noisy) 60 percent of Y
years = list(range(2000, 2021))
y = pd.Series([1000 * 1.03**t for t in range(len(years))], index=years)
noise = np.exp(np.random.default_rng(42).normal(0, 0.01, len(years)))
df_est = pd.DataFrame({'Y': y, 'CON': 0.6 * y * noise})

ls = Estimate_nls.with_defaults(
    smpl = (2001, 2020), # default estimation sample

```

```

input_df    = df_est,          # default data
param_names = ['lambda', 'beta_longterm', 'beta_shortterm'],
)

```

2.16.3 Estimation inside Makemodel

The tag `<stoc,estimator=ls>` asks `Makemodel` to estimate the equation with `ls` during construction. `<stoc>` marks the equation as behavioural; it is shorthand for `<exo>` (the variable can be fixed) **plus** `<add>` (an add factor is generated).

```

consumption = Makemodel(''
><stoc,estimator=ls> dlog(con) = -lambda*(log(con( -1)) -log(y( -1))
-log(abs(beta_longterm(1)))) + beta_shortterm(10)*dlog(y)
''')
consumption.show

FRML <STOC,ESTIMATOR=LS> CON = (CON( -1)*EXP(CON_A+ ( -(1.0885233961)*(LOG(CON( -1))
-LOG(Y( -1)) -LOG(ABS((0.5995680166))))+(1.0061077338)*((LOG(Y)) -(LOG(Y( -1)))))) *
(1 -CON_D)+ CON_X*CON_D $

FRML <CALC_ADD_FACTOR> CON_A = - (( -(1.0885233961)*(LOG(CON( -1)) -LOG(Y( -1))
-LOG(ABS((0.5995680166))))+(1.0061077338)*((LOG(Y)) -(LOG(Y( -1)))))) +LOG(CON)
-LOG(CON( -1)) $

```

2.16.4 Estimation tags

These options are placed inside the `<...>` tag of an equation:

Tag	Effect
<code><estimator=name></code> (alias <code><est=name></code>)	Estimate this equation with the estimator name — a factory made by <code>with_defaults</code> , an estimator class, or a name resolved through <code>estimator_classes=</code> or the caller's namespace
<code><estimator></code> / <code><est></code>	Estimate with the model-wide default passed as <code>Makemodel(..., estimator=...)</code>
<code><stoc></code>	Mark the equation as behavioural: implies <code><exo></code> and <code><add></code>
<code><smpl=start end></code>	Equation-local estimation sample; overrides the estimator and <code>Makemodel</code> defaults
<code><caption='text'></code>	Caption used for the equation in estimation output and reports
<code><constraints=...></code>	Coefficient constraints, appended to the equation as an ST. clause
<code><drop></code>	Skip the equation when the FRMLs are emitted (the text is kept in the documentation)

2.16.5 Options for Estimate_nls / .with_defaults()

Option	Default	Description
org_eq	—	The equation (positional when calling the factory)
input_df	None	DataFrame with the data; time on the index
smpl	(2002, 2018)	Estimation sample, inclusive
est_param	'C'	Prefix for numbered coefficients C(1), C(2), ...
param_names	None	Additional named coefficient prefixes, e.g. ['lambda', 'beta_longterm']
caption	'Estimation of '	Caption used in reports
var_description	{}	{variable: description} used to label output
constraints	{}	Per-parameter lmfit settings, e.g. {'C__1': {'min': 0, 'max': 1}}; also filled from an inline ST. clause
default_params	{}	Per-parameter starting values and bounds, e.g. {'C__2': {'value': -0.1, 'min': -1.0, 'max': 0.0}}
method	'least_squares'	Minimizer method passed to <code>lmfit.minimize</code>
fit_kws	{}	Extra keyword arguments forwarded to <code>lmfit.minimize</code>
coef_dict	{}	Numeric substitutions applied to placeholders before parsing
frml_name	'<STOC,DAMP>'	FRML tag used when the estimated equation is emitted
add_add_factor	True	Generate an add factor in the normalized equation
make_fixable	True	Generate <code>_X/_D</code> fixing variables
make_fitted	False	Generate a <code>_FITTED</code> equation

2.16.6 Imposing constraints — the ST. clause

Constraints on coefficients can be written directly in the equation after the keyword `ST.`, separated by `;`:

```
y = c(1) + c(2)*x + alfa*z st. c(1)~0.5; c(2)>0; alfa:=c(1)+c(2)
```

Form	Meaning
NAME = value	Fix the coefficient at <code>value</code> (not estimated)
NAME ~ value	Starting value (still estimated)
NAME >x / NAME >= x	Lower bound
NAME <x / NAME <= x	Upper bound
NAME = [lo, hi]	Lower and upper bound
NAME := expr	Derived coefficient: an algebraic function of other coefficients

Below the speed of adjustment is fixed at 0.3 instead of being estimated:

```
Makemodel('''
><stoc,estimator=ls> dlog(con) = -lambda*(log(con( -1)) -log(y( -1))
-log(abs(beta_longterm(1)))) + beta_shortterm(10)*dlog(y) st. lambda=0.3
''').show
```

→

```

FRML <STOC,ESTIMATOR=LS> CON = (CON( -1)*EXP(CON_A+ ( -(0.3000000000)*(LOG(CON( -1))
-LOG(Y( -1)) -LOG(ABS((0.6283287268))))+(0.5201481136)*((LOG(Y)) -(LOG(Y( -1)))))) *
(1 -CON_D)+ CON_X*CON_D $

FRML <CALC_ADD_FACTOR> CON_A = - (( -(0.3000000000)*(LOG(CON( -1)) -LOG(Y( -1))
-LOG(ABS((0.6283287268))))+(0.5201481136)*((LOG(Y)) -(LOG(Y( -1)))))) +LOG(CON
-LOG(CON( -1)) $

```

2.16.7 Estimation output and reports

- The `%%Makemymodel` magic renders the estimation result (coefficients, t-statistics, fit measures) below the cell.
- `.render_est` renders the model text with the estimation tables in the notebook.
- `.estimation_report(open_file=True)` writes an HTML report with actual-versus-fitted plots for every estimated equation; `report_all=True` also includes the identities.
- `.report().show()` displays the full model documentation with collapsible estimation panels.
- `.tvalues` returns the t-statistics of all estimated equations as a `DataFrame`.
- `.init_addfactors(df, start, end)` computes the add factors that make each behavioural equation reproduce the data.

The supplement chapter *Estimation* goes through estimation in detail.

2.17 %%Makemymodel — a Jupyter magic wrapper for Makemodel

Using Jupyter notebooks it can be useful to let the test, equations and lists be defines in a whole cell.

- `%%Makemymodel` is a Jupyter **cell magic** that creates a `Makemodel` instance from Markdown-based model text.
- `%%Makemymodel` is the corresponding **line magic**, used to assemble a model from its segments.

```

%%Makemymodel <name> [options]
text ...
> equation
>> continuation
more text and equations

```

The cell content is used to create a `Makemodel` instance.

- If **segments are not used**, the instance is named `<name>`
- If **segments are used**, the instance is named after the current `<segment name>`

2.17.1 Segments

Segments allow you to experiment with parts of a model independently.

The full model can later be assembled using the line magic:

```

%%Makemymodel <name> [options]

```

If

- `<segment name>` starts with `list`, the lists defined in the segment are used for all segments with the same `<name>`

2.17.2 Rules -

- `>` starts a model equation
- `»` continues the previous equation
- All other text is treated as Markdown

2.17.3 Common options

Option	Meaning
render	Render the model as Markdown
show	Print normalized equations
draw	Draw the dependency graph
segment=<segment name>	Store the cell as a named model segment

2.17.4 Summary

Markdown for humans, > for models.

```
%%Makemymodel test show
><add> a = log(b)

><add> a = log(b)

FRML <ADD> A = (log(B) + A_A) $

FRML <CALC_ADD_FACTOR> A_A = (A) - (log(B)) $

%%Makemymodel test show
><add> diff(a) = b

><add> diff(a) = b

FRML <ADD> A = A_A+ (B) +A( -1) $

FRML <CALC_ADD_FACTOR> A_A = A - ((B)) -A( -1) $

%%Makemymodel test show
><add> log(a) = b

><add> log(a) = b

FRML <ADD> A = EXP(A_A+ (B) ) $

FRML <CALC_ADD_FACTOR> A_A = - ((B)) +LOG(A) $

%%Makemymodel test show
><add> dlog(a) = b

><add> dlog(a) = b

FRML <ADD> A = A( -1)*EXP(A_A+ (B) ) $

FRML <CALC_ADD_FACTOR> A_A = - ((B)) +LOG(A) -LOG(A( -1)) $

%%Makemymodel test show
><add> log(a+c) = b

><add> log(a+c) = b

FRML <ADD> A = -C+EXP(A_A+ (B) ) $

FRML <CALC_ADD_FACTOR> A_A = - ((B)) +LOG(A+C) $

%%Makemymodel test show
><add,implicit,endo=price> demand = supply

><add,implicit,endo=price> demand = supply
```

```
FRML <ADD,IMPLICIT,ENDO=PRICE> PRICE___RES = ( SUPPLY ) - ( DEMAND ) $
```

```
%%Makemymodel test show
```

```
><exo,add,fit> a = b
```

```
><exo,add,fit> a = b
```

```
FRML <EXO,ADD,FIT> A = (B + A_A)* (1 -A_D)+ A_X*A_D $
```

```
FRML <FIT> A_FITTED = B $
```

```
FRML <CALC_ADD_FACTOR> A_A = (A) - (B) $
```

Chapter 3

The %%Makemymodel Jupyter Magic

The %%Makemymodel cell magic lets you write **model specifications directly in notebook cells** using Markdown-style text mixed with ModelFlow DSL equations. Under the hood it parses the cell, feeds everything to `Makemodel`, and pushes the resulting object into the notebook namespace — ready to use.

This notebook explains how it works and walks through all the options.

```
%load_ext autoreload
%autoreload 2

import modeljupytermagic          # registers the magics
from modelconstruct_estimation import Makemodel
```

3.1 1. Basic syntax

The first line after %%Makemymodel carries the **model name** and optional **flags**:

```
%%Makemymodel <name> [option1] [option2=value] ...
```

The rest of the cell is **Markdown text** where lines starting with > are treated as ModelFlow DSL equations. Everything else is documentation that gets rendered in the notebook.

After execution the `Makemodel` instance is available as a variable with the given <name>.

3.1.1 Minimal example

The cell below creates a `Makemodel` object called `simple` and renders the Markdown text in the notebook.

```
%%Makemymodel simple
A simple two -equation model
> y = c + i
> c = 0.8 * y(-1)

A simple two -equation model
```text
> y = c + i
> c = 0.8 * y(-1)
```
```

The variable `simple` is now a `Makemodel` instance in the notebook namespace. We can inspect it:

```
# Show the expanded equations
simple.show

FRML <> Y = C+I $
FRML <> C = 0.8*Y(-1) $

# Draw the causal graph
simple.draw
```

3.2 2. Available options

Options are placed on the first line after the model name. They follow the pattern **key** (boolean flag) or **key=value**.

| Option | Default | Effect |
|--|----------|---|
| <code>show</code> | off | Print the expanded equations after creation |
| <code>draw</code> | off | Display the causal dependency graph |
| <code>display</code> | off | Verbose mode: render Markdown and print equations + segment info |
| <code>render_est</code> | on | Render the Markdown text with estimation results included |
| <code>render</code> | on | Render the plain Markdown text (only consulted when <code>render_est=0</code>) |
| <code>render_list</code> | on | Include list definitions when rendering (set to 0 to hide them) |
| <code>latex</code> | off | Generate a LaTeX PDF of the model specification |
| <code>spec</code> | markdown | Rendering format (markdown by default) |
| <code>segment</code> | off | Segmented model building (see Section 4) |
| <code>replacements</code> | none | Dimension placeholders, e.g. <code>replacements=('__d','__-{banks}')</code> |
| <code>funks</code> | none | User-defined functions to pass to <code>Makemodel</code> |
| <code>estimator</code> (alias <code>est</code>) | none | Default estimator for equations tagged <code><est>/<estimator></code> without a value (see Section 6) |
| <code>input_df</code> | none | <code>DataFrame</code> used when tagged equations are estimated |
| <code>smpl</code> | none | Model-wide default estimation sample, e.g. <code>smpl="(2012, 2019)"</code> |

Values that are not literals are looked up as names in the notebook namespace, so `input_df=np1` or `estimator=ls` refer to notebook variables. Boolean options can be turned off explicitly with `option=0` or `option=False`.

3.2.1 Example: show and draw

```
%%Makemodel demo show draw
A model with tags
><exo,add> y = c + i
> c = 0.8 * y(-1)
```

```
A model with tags
```text
><exo,add> y = c + i
> c = 0.8 * y(-1)
```
```

```
FRML <EXO,ADD> Y = (C+I + Y_A)* (1 -Y_D)+ Y_X*Y_D $
FRML <> C = 0.8*Y(-1) $
```

```
FRML <CALC_ADD_FACTOR> Y_A = (Y) - (C+I) $
```

3.2.2 Example: suppressing rendering with render=0

```
%%Makemymodel quiet render=0 show
This text will NOT appear in the output
> a = b + c
```

```
This text will NOT appear in the output
```text
> a = b + c
```
```

```
FRML <> A = B+C $
```

3.3 3. Using lists and doable inside the magic

The cell content is passed directly to Makemodel, so all DSL features — lists, `do...enddo`, `doable`, `sum()`, tags — work exactly as they would in a normal `Makemodel()` call.

```
%%Makemymodel bank_model show
## Credit risk model
```

Define the banks and portfolios:

```
>list banks = banks : Danske Nordea
>list ports = ports : Mortgage Corporate
```

Calculate expected loss per bank and portfolio, then aggregate:

```
>doable <sum=total> el_{banks}_{ports} = exposure_{banks}_{ports} *
pd_{banks}_{ports}
```

↪

```
## Credit risk model
```

Define the banks and portfolios:

```
```text
>list banks = banks : Danske Nordea
>list ports = ports : Mortgage Corporate
```
```

Calculate expected loss per bank and portfolio, then aggregate:

```
```text
>doable <sum=total> el_{banks}_{ports} = exposure_{banks}_{ports} *
pd_{banks}_{ports}
```
```

↪

```
FRML <SUM=TOTAL> EL_DANSKE_MORTGAGE = EXPOSURE_DANSKE_MORTGAGE*PD_DANSKE_MORTGAGE $
FRML <SUM=TOTAL> EL_DANSKE_CORPORATE =
EXPOSURE_DANSKE_CORPORATE*PD_DANSKE_CORPORATE $
FRML <SUM=TOTAL> EL_NORDEA_MORTGAGE = EXPOSURE_NORDEA_MORTGAGE*PD_NORDEA_MORTGAGE $
FRML <SUM=TOTAL> EL_NORDEA_CORPORATE =
EXPOSURE_NORDEA_CORPORATE*PD_NORDEA_CORPORATE $
FRML <SUM=TOTAL> EL_TOTAL = ((EL_DANSKE_MORTGAGE+EL_DANSKE_CORPORATE)+(EL_NORDEA_
A_MORTGAGE+EL_NORDEA_CORPORATE))) $
```

↪

↪

↪

3.4 4. Segmented models

Large models can be split across **multiple cells** using the `segment=<name>` option. This is one of the most powerful features of the magic.

How it works:

1. Each cell with `segment=<name>` stores its content in a dictionary called `<modelname>_dict`.
2. Segments whose name starts with `list` or `text` are treated as definitions/documentation — they are stored but only rendered, not yet compiled.
3. When a non-list/non-text segment is encountered, the magic combines **all list segments** with the **current cell** and builds the `Makemodel` model.
4. A final call **without segment** combines **all** stored segments into the complete model.

This lets you build a model incrementally, cell by cell, with explanatory text between sections.

A practical naming convention: call the `Makemodel` instance `M<name>` (e.g. `Mtoymodel`) so the plain name stays free for the solvable model instance, extracted after the combining call with `toymodel = Mtoymodel.mmodel` — see Section 6.

```
%%Makemymodel stress segment=listtests
## List definitions
>list banks = banks : Alpha Beta
>list ports = ports : Retail SME
```

```
## List definitions
```text
>list banks = banks : Alpha Beta
>list ports = ports : Retail SME
```
```

```
%%Makemymodel stress segment=pd_block show
## PD dynamics
```

Probability of default follows an AR(1) process driven by GDP growth:

```
>doable <> pd_{banks}__{ports} = 0.02 + 0.8 * pd_{banks}__{ports}(-1) + 0.1 *
gdp_growth
```

→

```
## List definitions
```text
>list banks = banks : Alpha Beta
>list ports = ports : Retail SME
```
```

```
## PD dynamics
```

Probability of default follows an AR(1) process driven by GDP growth:

```
```text
>doable <> pd_{banks}__{ports} = 0.02 + 0.8 * pd_{banks}__{ports}(-1) + 0.1 *
gdp_growth
```
```

→

```
FRML <> PD__ALPHA__RETAIL = 0.02+0.8*PD__ALPHA__RETAIL(-1)+0.1*GDP_GROWTH $
FRML <> PD__ALPHA__SME = 0.02+0.8*PD__ALPHA__SME(-1)+0.1*GDP_GROWTH $
FRML <> PD__BETA__RETAIL = 0.02+0.8*PD__BETA__RETAIL(-1)+0.1*GDP_GROWTH $
FRML <> PD__BETA__SME = 0.02+0.8*PD__BETA__SME(-1)+0.1*GDP_GROWTH $
```

```
%%Makemymodel stress segment=loss_block show
```

```
## Expected loss
```

```
>doable <sum=total> el_{banks}__{ports} = exposure_{banks}__{ports} *
pd_{banks}__{ports}
```

→

```
## List definitions
```text
>list banks = banks : Alpha Beta
>list ports = ports : Retail SME
```
```

```
## Expected loss
```

```
```text
>doable <sum=total> el_{banks}__{ports} = exposure_{banks}__{ports} *
pd_{banks}__{ports}
```
```

→

```

FRML <SUM=TOTAL> EL__ALPHA__RETAIL = EXPOSURE__ALPHA__RETAIL*PD__ALPHA__RETAIL $
FRML <SUM=TOTAL> EL__ALPHA__SME = EXPOSURE__ALPHA__SME*PD__ALPHA__SME $
FRML <SUM=TOTAL> EL__BETA__RETAIL = EXPOSURE__BETA__RETAIL*PD__BETA__RETAIL $
FRML <SUM=TOTAL> EL__BETA__SME = EXPOSURE__BETA__SME*PD__BETA__SME $
FRML <SUM=TOTAL> EL__TOTAL =
((EL__ALPHA__RETAIL+EL__ALPHA__SME)+(EL__BETA__RETAIL+EL__BETA__SME)) $

```

Now combine all segments into the full model by calling `%%Makemymodel` (or `%Makemymodel` as a line magic) **without** the `segment` option:

```

%%Makemymodel stress show draw
## Full stress test model
Combining all segments into one model.

```

```

## PD dynamics
Probability of default follows an AR(1) process driven by GDP growth:
```text
>doable <> pd_{banks}_{ports} = 0.02 + 0.8 * pd_{banks}_{ports}(-1) + 0.1 *
gdp_growth
```

```

```

## Expected loss
```text
>doable <sum=total> el_{banks}_{ports} = exposure_{banks}_{ports} *
pd_{banks}_{ports}
```

```

```

## Full stress test model
Combining all segments into one model.

```

```

## List definitions
```text
>list banks = banks : Alpha Beta
>list ports = ports : Retail SME
```

```

```

FRML <> PD__ALPHA__RETAIL = 0.02+0.8*PD__ALPHA__RETAIL(-1)+0.1*GDP_GROWTH $
FRML <> PD__ALPHA__SME = 0.02+0.8*PD__ALPHA__SME(-1)+0.1*GDP_GROWTH $
FRML <> PD__BETA__RETAIL = 0.02+0.8*PD__BETA__RETAIL(-1)+0.1*GDP_GROWTH $
FRML <> PD__BETA__SME = 0.02+0.8*PD__BETA__SME(-1)+0.1*GDP_GROWTH $
FRML <SUM=TOTAL> EL__ALPHA__RETAIL = EXPOSURE__ALPHA__RETAIL*PD__ALPHA__RETAIL $
FRML <SUM=TOTAL> EL__ALPHA__SME = EXPOSURE__ALPHA__SME*PD__ALPHA__SME $
FRML <SUM=TOTAL> EL__BETA__RETAIL = EXPOSURE__BETA__RETAIL*PD__BETA__RETAIL $
FRML <SUM=TOTAL> EL__BETA__SME = EXPOSURE__BETA__SME*PD__BETA__SME $
FRML <SUM=TOTAL> EL__TOTAL =
((EL__ALPHA__RETAIL+EL__ALPHA__SME)+(EL__BETA__RETAIL+EL__BETA__SME)) $

```

3.5 5. Using replacements= for dimension placeholders

When variable names have many dimensions, you can define a replacement tuple in the notebook and pass it by name:

```

repl = ('__d', '__{banks}_{ports}')

```

```

%%Makemymodel compact show replacements=repl
>list banks = banks : A B
>list ports = ports : X Y
Compact notation using `__d` as placeholder:
>doable <> loss__d = exp__d * pd__d * lgd__d

```

```

```text
>list banks = banks : A B

```

```
>list ports = ports : X Y
...
Compact notation using `__d` as placeholder:
```text
>doable <> loss__d = exp__d * pd__d * lgd__d
...

FRML <> LOSS__A__X = EXP__A__X*PD__A__X*LGD__A__X $
FRML <> LOSS__A__Y = EXP__A__Y*PD__A__Y*LGD__A__Y $
FRML <> LOSS__B__X = EXP__B__X*PD__B__X*LGD__B__X $
FRML <> LOSS__B__Y = EXP__B__Y*PD__B__Y*LGD__B__Y $
```

3.6 6. Estimation inside the magic

Equations tagged with `<estimator=...>` are **estimated while the model is built** — the *Estimation* chapter tells the full story. The estimator name is looked up in the notebook namespace, so a factory created with `.with_defaults()` can be referenced directly in the tag. The magic renders the estimation results (coefficients, t-statistics, fit measures) below the cell; suppress this with `render_est=0`.

```
import numpy as np
import pandas as pd
from modelestimators_new import Estimate_nls

# A small synthetic dataset: CON is (noisy) 60 percent of Y
years = list(range(2000, 2021))
y = pd.Series([1000 * 1.03**t for t in range(len(years))], index=years)
noise = np.exp(np.random.default_rng(42).normal(0, 0.01, len(years)))
df_est = pd.DataFrame({'Y': y, 'CON': 0.6 * y * noise})

ls = Estimate_nls.with_defaults(smpl=(2001, 2020), input_df=df_est,
                                param_names=['lambda', 'beta_longterm', 'beta_shortterm'])

%%Makemymodel Mest segment=consumption
## Private consumption
><stoc,estimator=ls> dlog(con) = -lambda*(log(con( -1)) -log(y( -1)))
-log(abs(beta_longterm(1)))) + beta_shortterm(10)*dlog(y)

## Private consumption
```text
><stoc,estimator=ls> dlog(con) = -lambda*(log(con(-1)) -log(y(-1)))
-log(abs(beta_longterm(1)))) + beta_shortterm(10)*dlog(y)

Estimation output

| Item | Value |
|: - -: - -|
| Estimator | ls |
| Effective sample | (np.int64(2001), np.int64(2020)) |
| Original equation | DLOG(CON) = -LAMBDA*(LOG(CON(-1)) -LOG(Y(-1))
-LOG(ABS(BETA_LONGTERM(1)))) + BETA_SHORTTERM(10)*DLOG(Y) |
| Estimated equation | DLOG(CON) = -(1.0885233961)*(LOG(CON(-1)) -LOG(Y(-1))
-LOG(ABS((0.5995680166)))) + (1.0061077338)*DLOG(Y) |
| success | True |
| message | `ftol` termination condition is satisfied. |
| chisqr | 0.001417132888 |
| redchi | 8.33607581e -05 |
| aic | -185.0970363 |
| bic | -182.1098395 |
```

```
| Parameter | Estimate | t -stat |
|: - -| - -:| - -:|
| BETA_LONGTERM__1 | 0.5995680166 | |
| BETA_SHORTTERM__10 | 1.006107734 | |
| LAMBDA__1 | 1.088523396 | 4.536 |
```

A useful naming convention for estimated, segmented models: name the `Makemodel` instance `M<name>` — here `Mest` — and extract the solvable `modelclass.model` instance from the combining call with `.mmodel`:

```
%Makemymodel Mest render=False render_est=False
est = Mest.mmodel
```

The combined instance offers, among other things:

- `.render_est` — render the text plus estimation tables
- `.tvalues` — t-statistics for all estimated equations
- `.estimation_report(open_file=True)` — HTML report with actual-versus-fitted plots
- `.report().show()` — full model documentation with collapsible estimation panels
- `.init_addfactors(df, start, end, check=True)` — add factors so the model reproduces the data
- `.show<attribute>` — print any attribute nicely; an unknown name (e.g. `Mest.showclean`) triggers an error listing all the available show-properties

## 3.7 7. Tags inside the magic

All `Makemodel` tags work inside the magic cells. Here is a quick reference:

- `<exo>` — allow fixing the variable to an exogenous value
- `<add>` — generate add-factor adjustment variables
- `<fit>` — compute the original fitted value alongside the adjusted one
- `<sum=label>` — inside `doable`, also generate an aggregation equation
- `<add_suffix=__J>` — change the default add-factor suffix from `_A`
- `<stoc>` — mark the equation as behavioural: implies `<exo>` and `<add>`
- `<estimator=name>` / `<est=name>` — estimate the equation with the named estimator (see Section 6)
- `<smpl=start end>` — equation-local estimation sample
- `<caption='text'>` — caption used in estimation output and reports
- `<drop>` — leave the equation out of the emitted model (the text is kept)

Tags are placed between `><` and `>` at the start of a formula line:

```
%%Makemymodel tagged show
A fully decorated equation:
><exo,add,fit> consumption = 0.8 * income
```

```
A fully decorated equation:
```text
><exo,add,fit> consumption = 0.8 * income
```
```

```
FRML <EXO,ADD,FIT> CONSUMPTION = (0.8*INCOME + CONSUMPTION_A)* (1 -CONSUMPTION_D)+
CONSUMPTION_X*CONSUMPTION_D $
FRML <FIT> CONSUMPTION_FITTED = 0.8*INCOME $
FRML <CALC_ADD_FACTOR> CONSUMPTION_A = (CONSUMPTION) - (0.8*INCOME) $
```

↪

## 3.8 8. Generating a LaTeX PDF

With the `latex` option, the magic converts the cell's Markdown text (headings, tables, bullet lists, equation lines) into LaTeX and compiles a PDF. This requires a LaTeX installation on the system.

```
%%Makemymodel mymodel latex
My Model Documentation
> y = c + i
```

## 3.9 9. How it works internally

The magic is implemented in `modeljupytermagic.py`. Here is the flow:

1. **Parse the first line** — `get_options(line)` splits it into the model name and a dictionary of options. Values that are not literals are resolved as names in the notebook namespace (`_resolve_option`).
2. **Handle segments** — if `segment=` is present, the cell text is stored in `<name>_dict[segment]`. List/text segments are rendered and returned early. Other segments combine list definitions with the current cell.
3. **Build the model** — the combined text is passed to `Makemodel(model_text, replacements=..., funks=..., estimator=..., input_df=..., smpl=...)`. This does all DSL expansion: lists, loops, doable, sums, tags. The notebook namespace is passed as `estimator_namespace`, so `<estimator=name>` tags resolve to notebook variables and equations so tagged are estimated on the spot.
4. **Push to namespace** — the resulting `Makemodel` instance is injected into the notebook's user namespace via `ip.push()`.
5. **Render output** — depending on options, the magic renders Markdown with estimation results (`render_est`, the default) or plain (`render`), prints expanded equations (`.show`), draws the dependency graph (`.draw`), or generates LaTeX.

The key insight is that the magic is a **thin wrapper** — all the heavy lifting is done by `Makemodel`. The magic simply provides a convenient notebook-native interface for defining and documenting models in one place.

## 3.10 10. Quick reference

```
%%Makemymodel <name> [show] [draw] [display] [render=0] [render_est=0] [render_list=0]
 [latex] [segment=<seg>] [replacements=<var>] [funks=<var>]
 [estimator=<var>] [input_df=<var>] [smpl="(start, end)"]
<Markdown text with > prefixed DSL equations>
```

After execution: the variable `<name>` holds the `Makemodel` instance with all the usual properties — `.show`, `.draw`, `.render`, `.render_est`, `.tvalues`, `.estimation_report(...)`, `.report().show()`, `.init_addfactors(...)`, the `show<attribute>` printers, and `.mmodel` for the solvable `model-class.model` instance.

For segmented, estimated models the recommended pattern is:

```
%%Makemymodel Mtoymodel segment=consumption # one cell per segment
...
%Makemymodel Mtoymodel render=False render_est=False # combine
toymodel = Mtoymodel.mmodel # extract the solvable model
```

## Chapter 4

# Estimating behavioural equations

A ModelFlow model combines **identities**, which hold exactly, with **behavioural equations**, whose coefficients are estimated from data. Estimation is part of building the model: an equation tagged `<est=...>` is estimated while `Makemodel` — or the `%%Makemodel` magic — processes it, and the estimated coefficients are baked into the FRML it emits.

Three estimation methods are built in and can be named in the tag with nothing imported: `ols`, ordinary least squares through `statsmodels`, for equations linear in their parameters; `nls_lmfit`, non-linear least squares through the `lmfit` package, which is also the only backend accepting coefficient constraints; and `nls_eviews`, for those who have EViews. All three take the same equation syntax, so the choice is only about what the equation looks like.

An estimator can also be built in Python — with `.with_defaults()` — when it needs options the tag cannot carry. That comes later in the chapter; nothing before it needs an import.

### Note

This chapter is the **syntax**, worked on small synthetic data where the right answer is known in advance. A complete estimation on real data — a model for Nepal, estimated equation by equation, assembled and given add factors until it reproduces history — is a supplement of its own: *ModelFlow - Estimation, a worked example*.

```
import pandas as pd
import numpy as np

from modelclass import model
import modeljupytermagic # %%dataframe and %%Makemodel magics
from modelconstruct_estimation import Makemodel

Development support
%load_ext autoreload
%autoreload 2
```

### 4.1 A first example, where the answer is known

Before any real data, a synthetic set where the right answer is known in advance. The data below comes from a consumption function with coefficients we choose ourselves:

$$CON_t = 100 + 0.6 Y_t + \varepsilon_t \quad (4.1)$$

$Y$  grows at 3 per cent a year and  $\varepsilon$  is a relative disturbance with standard deviation `sd`. Estimating that same equation back out of the data shows what the estimator does — and turning `sd` down shows what the disturbance costs.

Nothing has to be imported to estimate. An equation tagged `<est=name>` is estimated while the model is built, and three method names are built in:

| Name       | Backend                   | For                                                   |
|------------|---------------------------|-------------------------------------------------------|
| ols        | statsmodels least squares | equations linear in their parameters                  |
| nls_lmfit  | lmfit least squares       | any equation; the only backend taking ST. constraints |
| nls_eviews | EViews                    | any equation, if EViews is installed                  |

The data, the sample and the coefficient names come from the %%Makemymodel line as `input_df=`, `smpl=` and `param_names=`.

```
def make_data(sd=0.02, years=range(2000, 2021), seed=42):
 '''Synthetic data from a known consumption function: CON = 100 + 0.6*Y

 sd is the standard deviation of the relative disturbance on CON.'''

 idx = list(years)
 y = pd.Series([1000 * 1.03**t for t in range(len(idx))], index=idx)
 con = 100 + 0.6 * y
 noise = np.random.default_rng(seed).normal(0, sd, len(idx))
 return pd.DataFrame({'Y': y, 'CON': con * (1 + noise)})

small = make_data()
small.head()
```

|      | Y          | CON        |
|------|------------|------------|
| 2000 | 1000.00000 | 704.266039 |
| 2001 | 1030.00000 | 703.065828 |
| 2002 | 1060.90000 | 747.594746 |
| 2003 | 1092.72700 | 769.850695 |
| 2004 | 1125.50881 | 745.052328 |

```
%%Makemymodel Mlinear input_df=small smpl=(2001,2020) param_names=['alfa','beta_longterm']
A linear consumption function
The long -run relation, estimated by ordinary least squares. The true coefficients are →
100 and 0.6:
><stoc,est=ols> con = alfa + beta_longterm*y
```

## 4.2 A linear consumption function

The long-run relation, estimated by ordinary least squares. The true coefficients are 100 and 0.6:

```
><stoc,est=ols> con = alfa + beta_longterm*y
```

### Estimation output

| Item               | Value                                    |
|--------------------|------------------------------------------|
| Estimator          | ols                                      |
| Requested sample   | (np.int64(2001), np.int64(2020))         |
| Effective sample   | (np.int64(2001), np.int64(2020))         |
| Original equation  | CON = ALFA + BETA_LONGTERM*Y             |
| Estimated equation | CON = (85.3489879901) + (0.6101288809)*Y |
| aic                | 168.1593835                              |
| bic                | 170.150848                               |
| R-squared          | 0.9896705862                             |
| Adj. R-squared     | 0.9890967298                             |

| Parameter        | Estimate     | t-stat |
|------------------|--------------|--------|
| ALFA__1          | 85.34898799  | 4.139  |
| BETA_LONGTERM__1 | 0.6101288809 | 41.528 |

The same relation can also be written as an error correction equation, which is the form a behavioural equation usually takes in a macro model: a long-run share the variable returns to, a short-run response to growth, and a speed  $\lambda$  at which the gap closes. It is not linear in its parameters, so it needs `nls_lmfit`.

Two things change with it. It goes in a model of its own, since two equations for `CON` cannot live in the same one. And the sample cannot start in 2000: the equation refers to `con(-1)` and `y(-1)`, which do not exist in the first row of the data, so the first year pays for the lag. Every estimation output reports the sample it actually used next to the one it was given.

```
%%Makemymodel Mecm input_df=small smpl=(2000,2020)
param_names=['lambda','beta_shortterm','beta_longterm']
Consumption as an error correction equation
><stoc,est=nls_lmfit> dlog(con) = -lambda*(log(con(-1)) -log(y(-1))
-log(abs(beta_longterm(1)))) + beta_shortterm(10)*dlog(y)
```

### 4.3 Consumption as an error correction equation

```
><stoc,est=nls_lmfit> dlog(con) = -lambda*(log(con(-1)) -log(y(-1))
-log(abs(beta_longterm(1)))) + beta_shortterm(10)*dlog(y)
```

#### Estimation output

| Item               | Value                                                                                                        |
|--------------------|--------------------------------------------------------------------------------------------------------------|
| Estimator          | nls_lmfit                                                                                                    |
| Requested sample   | (np.int64(2000), np.int64(2020))                                                                             |
| Effective sample   | (np.int64(2001), np.int64(2020))                                                                             |
| Original equation  | DLOG(CON) = -LAMBDA (LOG(CON(-1))-<br>LOG(Y(-1))-LOG(ABS(BETA_LONGTERM(1))))<br>+ BETA_SHORTTERM(10)DLOG(Y)  |
| Estimated equation | DLOG(CON) = -(0.6136030211)(LOG(CON(-<br>1))-LOG(Y(-1))-LOG(ABS((0.6949833169)))) +<br>(0.2955687108)DLOG(Y) |
| success            | True                                                                                                         |
| message            | ftol termination condition is satisfied.                                                                     |
| chisqr             | 0.008515199595                                                                                               |
| redchi             | 0.0005008940939                                                                                              |
| aic                | -149.232696                                                                                                  |
| bic                | -146.2454991                                                                                                 |

| Parameter          | Estimate     | t-stat |
|--------------------|--------------|--------|
| BETA_LONGTERM__1   | 0.6949833169 | 0.000  |
| BETA_SHORTTERM__10 | 0.2955687108 | 0.000  |
| LAMBDA__1          | 0.6136030211 | 2.860  |

Under each equation the magic prints what the estimation did: the effective sample, the equation before and after the coefficients were substituted, and each parameter with its t-statistic. `ALFA__1` and `BETA_LONGTERM__1` in the linear equation should land close to the 100 and 0.6 the data was built from — not on them, because the disturbance is in the way.

Writing `<est=ols>` on every equation gets tiring in a model of any size. A **model-wide default** is set with `estimator=` — either on the `%%Makemymodel` line or as a `Makemodel(...)` argument — and the equations then need only a bare `<est>`.

### 4.3.1 What the noise costs

The disturbance is the only thing standing between the estimator and the true coefficients. Generating the same data with less and less of it shows how much it costs. `Makemodel` is used directly here rather than through the magic, because a magic is one cell and this wants a loop — the estimator object of each equation is kept in `estimation_records`:

```
def estimate(sd):
 '''Estimate con = alfa + beta_longterm*y on data disturbed by sd.

 estimator='ols' is the model -wide default, so the equation needs only <est>.
 param_names travels in estimator_kwargs, which is what the magic line's
 param_names= option fills in for you.
 Returns the estimator object of the one estimated equation.'''

 mm = Makemodel('>est> con = alfa + beta_longterm*y',
 input_df=make_data(sd), smpl=(2000, 2020), estimator='ols',
 estimator_kwargs={'param_names': ['alfa', 'beta_longterm']})
 return mm.estimation_records[0]['estimator_object']

noise_levels = [0.10, 0.05, 0.02, 0.01, 0.001]
ests = {sd: estimate(sd) for sd in noise_levels}

true values: ALFA__1 = 100, BETA_LONGTERM__1 = 0.6
pd.DataFrame({sd: e.coef_ser for sd, e in ests.items()}).T.rename_axis('noise sd')
```

|          | ALFA__1   | BETA_LONGTERM__1 |
|----------|-----------|------------------|
| noise sd |           |                  |
| 0.100    | 46.434862 | 0.637758         |
| 0.050    | 73.217431 | 0.618879         |
| 0.020    | 89.286972 | 0.607552         |
| 0.010    | 94.643486 | 0.603776         |
| 0.001    | 99.464349 | 0.600378         |

```
pd.DataFrame({sd: e.tvalues for sd, e in ests.items()}).T.rename_axis('noise sd')
```

|          | ALFA__1    | BETA_LONGTERM__1 |
|----------|------------|------------------|
| noise sd |            |                  |
| 0.100    | 0.493270   | 9.397386         |
| 0.050    | 1.555554   | 18.238412        |
| 0.020    | 4.742407   | 44.761490        |
| 0.010    | 10.053828  | 88.966621        |
| 0.001    | 105.659411 | 884.658965       |

As `sd` falls the coefficients close on 100 and 0.6 and the t-statistics grow. At the bottom of the table the equation is recovered almost exactly, which is the useful check: the estimator is not doing anything to the data beyond what the disturbance forces it to.

Outside a textbook the noise is not a knob. What the experiment shows is what to reach for when estimates come out imprecise, because everything that buys precision does it by improving the same ratio between signal and disturbance:

|                        | How it helps                                                                                                                                        |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| A longer sample        | More observations per parameter. <code>smpl=</code> is the first thing to widen — if the extra years are comparable                                 |
| A better specification | A term that genuinely belongs moves variance out of the residual and into the equation                                                              |
| Fewer parameters       | Each one is estimated from the same data; dropping one that the data cannot identify sharpens the rest                                              |
| Constraints            | Fixing or bounding a coefficient you already know spends information you have instead of asking the data for it — the <code>ST.</code> clause below |

The last one is the most useful when the sample is short, which in macro it usually is. The worked example in *ModelFlow - Estimation, a worked example* is estimated on nine annual observations, which is an ordinary length for a national accounts model.

## 4.4 A named estimator with `.with_defaults()`

Everything so far has been supplied per cell — the data, the sample, the coefficient names, repeated on every magic line. When several equations share them, and especially when an estimation needs options a magic line cannot carry, build a **factory** once and refer to it by name.

`.with_defaults()` collects the options and returns a callable that constructs and runs an estimation for any equation it is given; any keyword passed to the callable, or any equation tag, overrides a stored default. This is also the one place where the estimator class itself has to be imported.

A factory in the namespace is used exactly like a built-in name — `<est=ls>` — since `Makemodel` resolves the tag against the calling namespace as well as the built-ins.

```
from modelestimator_new import Estimate_nls

ls = Estimate_nls.with_defaults(
 smpl = (2000, 2020),
 input_df = small,
 var_description = {'Y': 'Income', 'CON': 'Consumption'},
 param_names = ['lambda', 'beta_shortterm', 'beta_longterm'],
)

est = ls("dlog(con) = -lambda*(log(con(-1)) -log(y(-1)) -log(abs(beta_longterm(1)))) →
+ beta_shortterm(10)*dlog(y)",
 caption='Consumption, named coefficients')
est.coef_ser

BETA_LONGTERM__1 0.694983
BETA_SHORTTERM__10 0.295569
LAMBDA__1 0.613603
Name: Consumption, named coefficients, dtype: float64
```

This is the same equation as `Mecm` above, written with names instead of `c(1)`, `c(2)` and `c(10)`, so the estimates are the same and only the labels differ.

### 4.4.1 Inspecting the result

Displayed on its own, the estimator object renders a full report — regression results and actual against fitted. It also exposes the result piece by piece:

| Attribute                      | Content                                                                 |
|--------------------------------|-------------------------------------------------------------------------|
| <code>coef_ser</code>          | Estimated coefficients as a Pandas <b>Series</b>                        |
| <code>tvalues</code>           | t-statistics as a <b>Series</b> (NaN for fixed or derived coefficients) |
| <code>org_eq_baked</code>      | The equation with the estimated numbers substituted in place            |
| <code>af_df</code>             | Actual and fitted values over the estimation sample                     |
| <code>residuals_df</code>      | Residuals over the estimation sample                                    |
| <code>regression_model</code>  | The native <code>lmfit</code> result object                             |
| <code>get_html_report()</code> | The full report as an HTML string                                       |

`org_eq_baked` is the one to read first: it is the equation that will go into the model, and reading it back is the cheapest check that the equation estimated is the equation intended.

```
print(est.org_eq_baked)
est.tvalues
```

```
DLOG(CON) = -(0.6136030211)*(LOG(CON(-1)) -LOG(Y(-1)) -LOG(ABS((0.6949833169)))) + ↪
(0.2955687108)*DLOG(Y)
```

```
BETA_LONGTERM__1 2.439227e -06
BETA_SHORTTERM__10 3.473041e -08
LAMBDA__1 2.859759e+00
Name: Consumption, named coefficients, dtype: float64
```

## 4.5 The full syntax of an estimation

Everything the estimator needs can be written into the equation string. The complete form is

```
[<tags>] lhs = rhs [ST. constraint ; constraint ; ...]
```

The tags are read by `Makemodel` and `%%Makemodel` — they are listed under *Estimation tags* below. The `ST.` clause is the subject of the next section. The rest is the equation itself.

Case is irrelevant: the whole string is upper-cased before parsing, so `dlog(con)` and `DLOG(CON)` are the same equation.

### 4.5.1 Left hand side

The endogenous variable, optionally transformed — `con`, `log(con)`, `dlog(con)`, `diff(con)`.

With `ecm=True`, the default for the NLS backends, the left hand side **must** be `dlog(var)`: the equation is read as an error-correction form, and `c(2)` — if it is present — is treated as the speed of adjustment. For a plain level or log equation use `ols`, which sets `ecm=False`. That is why `con = alfa + beta_longterm*y` is accepted above and would be rejected by `nls_lmfit`.

### 4.5.2 Coefficients

A coefficient is a prefix and a number:

| Written                                                  | Becomes                                                  | Where the prefix comes from                                 |
|----------------------------------------------------------|----------------------------------------------------------|-------------------------------------------------------------|
| <code>c(1)</code> , <code>c(2)</code>                    | <code>C__1</code> , <code>C__2</code>                    | <code>est_param</code> , which defaults to <code>'C'</code> |
| <code>lambda(1)</code> , <code>beta_shortterm(10)</code> | <code>LAMBDA__1</code> , <code>BETA_SHORTTERM__10</code> | a name declared in <code>param_names</code>                 |
| <code>lambda</code>                                      | <code>LAMBDA__1</code>                                   | a bare name is number 1 of that prefix                      |

Named prefixes are worth the small effort: a coefficient called `LAMBDA` is checked against economic sense more easily than one called `C__2`. Declare them with `param_names=` — on the `%%Makemodel` line, as a `Makemodel(..., estimator_kwargs={'param_names': [...]} )` argument, or stored once in a `.with_defaults()` factory.

The numbers are labels, not positions — `beta_shortterm(10)` beside `beta_longterm(1)` is perfectly ordinary, and the gaps are what make the numbering useful for keeping a coefficient’s identity stable while an equation is being reworked. Setting `est_param='B'` renames the anonymous prefix, and then both `C(1)` and `B(1)` mean `B__1`.

This is also the one syntactic trap in the language. A name with a number after it is a coefficient **only** if its prefix was declared; otherwise `alfa(1)` reads as a *lead* of a variable called `ALFA`. The check below catches it, but the error talks about an unknown variable rather than a missing `param_names` entry.

### 4.5.3 Right hand side

Ordinary ModelFlow expressions: arithmetic, lags such as `y(-1)`, and functions — `log`, `dlog`, `diff`, `exp`, `abs`. Each side must parse as Python, and a missing operator is reported with its position.

Every name which is not a declared coefficient has to be a column in `input_df`. An unknown one raises rather than entering the estimation as an all-NaN column, which is worth having: silently fitting against a column of NaN is the failure that produces a plausible-looking, meaningless coefficient vector.

### 4.5.4 Which backend does what

|                     | ordinary least squares                                                                                         | non-linear least squares          | EViews                             |
|---------------------|----------------------------------------------------------------------------------------------------------------|-----------------------------------|------------------------------------|
| Written in a tag as | <code>&lt;est=ols&gt;</code>                                                                                   | <code>&lt;est=nlslmfit&gt;</code> | <code>&lt;est=nlseviews&gt;</code> |
| Class               | <code>Estimate_ols</code>                                                                                      | <code>Estimate_nlslmfit</code>    | <code>Estimate_nlseviews</code>    |
| Engine              | <code>statsmodels</code>                                                                                       | <code>lmfit</code>                | <code>EViews</code>                |
| Equation            | linear in the parameters                                                                                       | any                               | any                                |
| ecm default         | <code>False</code>                                                                                             | <code>True</code>                 | <code>True</code>                  |
| ST. constraints     | not supported                                                                                                  | supported                         | not supported                      |
| Intercept           | write it explicitly, as <code>alfa</code> or <code>c(1)</code> ; omitting it fits through the origin and warns | as written                        | as written                         |

`Estimate_nls` is a factory over the last two: it never produces an instance of itself, but returns the backend named by `solver='lmfit'` or `solver='eviews'`. All of the classes carry `.with_defaults()`, and a factory built from any of them can be named in a tag just like a built-in.

## 4.6 Constraints — the ST. clause

Coefficients can be constrained directly in the equation string after the keyword `ST.`, with individual constraints separated by `;`:

```
| y = c(1) + c(2)*x + alfa*z st. c(1)~0.5; c(2)>0; alfa:=c(1)+c(2)
```

| Form                                   | Meaning                                                          |
|----------------------------------------|------------------------------------------------------------------|
| <code>NAME = value</code>              | Fix the coefficient at <code>value</code> (not estimated)        |
| <code>NAME ~ value</code>              | Starting value (still estimated)                                 |
| <code>NAME &gt;x / NAME &gt;= x</code> | Lower bound                                                      |
| <code>NAME &lt;x / NAME &lt;= x</code> | Upper bound                                                      |
| <code>NAME = [lo, hi]</code>           | Lower and upper bound                                            |
| <code>NAME := expr</code>              | Derived coefficient: an algebraic function of other coefficients |

The same constraints can be supplied programmatically through the `constraints=` option as per-parameter `lmfit` settings, e.g. `constraints={'LAMBDA__1': {'value': 0.3, 'vary': False}}`.

Below the speed of adjustment  $\lambda$  is fixed at 0.3, so only the two  $\beta$ s are estimated:

```
est_fix = ls("dlog(con) = -lambda*(log(con(-1)) -log(y(-1))
-log(abs(beta_longterm(1)))) + beta_shortterm(10)*dlog(y) st. lambda = 0.3",
 caption='Consumption, lambda fixed')
est_fix.coef_ser
```

```
BETA_LONGTERM__1 0.884157
BETA_SHORTTERM__10 -1.853228
LAMBDA__1 0.300000
Name: Consumption, lambda fixed, dtype: float64
```

And here  $\lambda$  is restricted to the interval  $[0.1, 0.5]$  while the short-run elasticity is kept non-negative:

```
est_bound = ls("dlog(con) = -lambda*(log(con(-1)) -log(y(-1))
-log(abs(beta_longterm(1)))) + beta_shortterm(10)*dlog(y) st. lambda = [0.1, 0.5];
beta_shortterm(10) >= 0",
 caption='Consumption, bounded')
est_bound.coef_ser
```

```
BETA_LONGTERM__1 0.401768
BETA_SHORTTERM__10 9.672186
LAMBDA__1 0.500000
Name: Consumption, bounded, dtype: float64
```

## 4.7 Options reference

All options accepted by `Estimate_nls` — directly or through `.with_defaults()`:

| Option                       | Default          | Description                                                                                                                |
|------------------------------|------------------|----------------------------------------------------------------------------------------------------------------------------|
| <code>org_eq</code>          | —                | The equation (positional when calling the factory)                                                                         |
| <code>input_df</code>        | None             | <code>DataFrame</code> with the data; time on the index                                                                    |
| <code>smpl</code>            | (2002, 2018)     | Estimation sample, inclusive                                                                                               |
| <code>est_param</code>       | 'C'              | Prefix for numbered coefficients <code>C(1)</code> , <code>C(2)</code> , ...                                               |
| <code>param_names</code>     | None             | Additional named coefficient prefixes, e.g. <code>['lambda', 'beta_longterm']</code>                                       |
| <code>caption</code>         | 'Estimation of ' | Caption used in reports                                                                                                    |
| <code>var_description</code> | {}               | <code>{variable: description}</code> used to label output                                                                  |
| <code>constraints</code>     | {}               | Per-parameter lmfit settings, e.g. <code>{ 'C__1': { 'min': 0, 'max': 1 } }</code> ; also filled from an inline ST. clause |
| <code>default_params</code>  | {}               | Per-parameter starting values and bounds, e.g. <code>{ 'C__2': { 'value': -0.1, 'min': -1.0, 'max': 0.0 } }</code>         |
| <code>method</code>          | 'least_squares'  | Minimizer method passed to <code>lmfit.minimize</code>                                                                     |
| <code>fit_kws</code>         | {}               | Extra keyword arguments forwarded to <code>lmfit.minimize</code>                                                           |
| <code>coef_dict</code>       | {}               | Numeric substitutions applied to placeholders before parsing                                                               |
| <code>frml_name</code>       | '<STOC,DAMP>'    | FRML tag used when the estimated equation is emitted                                                                       |
| <code>add_add_factor</code>  | True             | Generate an add factor in the normalized equation                                                                          |
| <code>make_fixable</code>    | True             | Generate <code>_X/_D</code> fixing variables                                                                               |
| <code>make_fitted</code>     | False            | Generate a <code>_FITTED</code> equation                                                                                   |

Every parameter without an explicit starting value starts at 0.1. For ECM equations the speed-of-adjustment often needs help: give it a starting value or bounds through `default_params` or an `ST.` clause.

## 4.8 Estimation while building a model

Estimation really shines inside `Makemodel`: an equation tagged with `<estimator=...>` is estimated during model construction, and the estimated coefficients are baked into the emitted `FRML`. The name is resolved first against an explicit `estimator_classes=` mapping, then against the calling namespace — which is how a factory such as `ls` above is found — and last against the built-in methods `ols`, `nls_lmfit` and `nls_eviews`. A variable in the notebook therefore shadows a built-in of the same name.

### 4.8.1 Estimation tags

| Tag                                                                        | Effect                                                                                                                                 |
|----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <code>&lt;estimator=name&gt;</code> (alias <code>&lt;est=name&gt;</code> ) | Estimate this equation with the estimator <code>name</code> — a factory in the namespace or a built-in method                          |
| <code>&lt;estimator&gt;</code> / <code>&lt;est&gt;</code>                  | Estimate with the model-wide default passed as <code>Makemodel(..., estimator=...)</code> or <code>estimator=</code> on the magic line |
| <code>&lt;stoc&gt;</code>                                                  | Mark the equation as behavioural: implies <code>&lt;exo&gt;</code> (fixable) and <code>&lt;add&gt;</code> (add factor)                 |
| <code>&lt;smpl=start end&gt;</code>                                        | Equation-local estimation sample; overrides the estimator and <code>Makemodel</code> defaults                                          |
| <code>&lt;caption='text'&gt;</code>                                        | Caption used for the equation in estimation output and reports                                                                         |
| <code>&lt;constraints=...&gt;</code>                                       | Coefficient constraints, appended to the equation as an <code>ST.</code> clause                                                        |
| <code>&lt;drop&gt;</code>                                                  | Skip the equation when the <code>FRMLs</code> are emitted (the text is kept in the documentation)                                      |

An equation tagged `<stoc>` comes back from construction **normalized** — solved for its endogenous variable rather than for a log difference — with an add factor and a fixing pair attached, and with a `<CALC_ADD_FACTOR>` equation generated alongside it. *ModelFlow - Estimation, a worked example* builds a complete model this way and reads the emitted `FRMLs`.

### 4.8.2 Estimation output and reports

A `Makemodel` built from estimated equations keeps the estimations, not only their results:

- `.render_est` renders the model text with estimation tables (coefficients, t-statistics, fit measures) — the `%%Makemymodel` magic does this automatically.
- `.tvalues` collects the t-statistics of all estimated equations in one `DataFrame`.
- `.estimation_report(open_file=True)` writes an HTML report with actual-versus-fitted plots for every estimated equation; `report_all=True` also includes the identities.
- `.report().show()` displays the full model documentation — prose, equations and collapsible estimation panels.
- `.estimation_records` holds the raw record of every estimation run during construction, including the estimator object itself; `.showestimation_records` prints it.

## 4.9 Quick reference

```
| [<tags>] lhs = rhs [ST. constraint ; constraint ; ...] # the full equation form
```

Nothing imported — the built-in method names, with the data, sample and coefficient names on the magic line:

```
%%Makemymodel Mdemo input_df=df smpl=(2000,2020) estimator=ols →
param_names=['alfa','beta_longterm']
Consumption
><stoc,est> con = alfa + beta_longterm*y
Investment
><stoc,est=nls_lmfit> dlog(inv) = -lambda*(log(inv(-1)) -log(y(-1)) →
-log(abs(beta_longterm(1)))) + beta_shortterm(10)*dlog(y)
```

A factory, when several equations share the same options:

```
from modelestimatorm_new import Estimate_nls

ls = Estimate_nls.with_defaults(smpl=(start, end), input_df=df,
 param_names=['lambda', 'beta_shortterm', 'beta_longterm'],
 var_description={...})

est = ls("dlog(x) = -lambda*(log(x(-1)) -log(y(-1)) -log(abs(beta_longterm(1))))"
 " + beta_shortterm(10)*dlog(y) st. lambda=[0.1, 0.5]")
est.coef_ser; est.tvalues; est.org_eq_baked
```

Building and using the model:

```
mm = Makemodel("><stoc,est=ls,caption='...',smpl=2012 2019> dlog(x) = ...")
mm.render_est; mm.tvalues; mm.estimation_report(open_file=True); mm.report().show()
mm.estimation_records[0]['estimator_object'] # the estimator of one equation
solvable = mm.mmodel
baseline = mm.init_addfactors(df, start, end, check=True)
```

## Chapter 5

# The normal function

The function `normal` from the module `modelnormalize` **normalizes** model equations: it takes a business-logic equation written as `lhs = rhs` and rewrites it so that the equation's endogenous variable stands alone on the left-hand side. In a model each equation determines one endogenous variable, so the normalized form of equation  $i$  is  $y_i = F_i(y, x)$  — the right-hand side may contain the other endogenous variables of the model and any lagged (or leaded) values, including lags of  $y_i$  itself; only the current-period  $y_i$  must not appear on the right. This is the form the ModelFlow solvers require.

Beyond isolating the endogenous variable, `normal` can:

- **preprocess** EViews-style transformation functions (`DLOG`, `DIFF`, `PCT_GROWTH`, `PCY`, `MOVAVG`, `LOGIT`, `D`, ...) into elementary expressions,
- **invert** a transformation applied to the endogenous variable on the left-hand side (for instance solve `dlog(y) = rhs` for `y`),
- inject an **add factor** (adjustment term) `<var>_A` together with the equation that calculates it from data,
- make the equation **fixable** (exogenizable) through a dummy `<var>_D` and a fixing value `<var>_X`,
- produce a **fitted** version of the equation, stripped of add factor and fixing terms,
- recast **implicit** equations (equilibrium conditions) as residual equations for the solver.

`normal` is the workhorse behind `Makemodel` and therefore also the `%Makemymodel` cell magic. It returns an instance of the class `Normalized_frml`, which holds the different variants of the equation.

This notebook walks through the main use cases. The reader is assumed to be comfortable with Python and with macroeconomic model equations.

## 5.1 Imports

Import the `normal` function and the `Normalized_frml` result class.

```
| from modelnormalize import normal, Normalized_frml
```

## 5.2 The result class: `Normalized_frml`

Every call to `normal` returns a `Normalized_frml` instance. Its attributes hold the different representations of the equation — original, preprocessed, normalized, the add-factor calculation, the fitted version and so on. Displaying the instance prints the attributes that are set.

```
| print(Normalized_frml.__doc__)
```

```
| Class defining the result from normalization of an expression.
```

```
| Attributes:
```

```
| endo_var (str): The endogenous variable.
```

```
| eviews (str): The EViews representation of the expression.
```

```
| original (str): The original Business logic expression.
```

```

preprocessed (str): The preprocessed expression.
normalized (str): The normalized expression.
calc_add_factor (str): The calculated additional factor.
un_normalized (str): The un -normalized expression.
fitted (str): The fitted expression.

```

### 5.3 The signature of normal

The docstring below lists all arguments. The ones used in this notebook are `the_endo`, `add_add_factor`, `add_suffix`, `endo_lhs`, `make_fixable`, `make_fitted` and `implicit`.

```

print(normal.__doc__)

normalize an expression g(y,x) = f(y,x) ==> y = F(x,z)

Default find the expression for the first variable on the left hand side (lhs)

The variable - without lags - should not be on rhs.

Args:
 ind_o (str): input expression, no $ and no frml name just lhs=rhs
 the_endo (str, optional): the endogeneous to isolate on the left hans side. if the →
 first variable in the lhs.
 It shoud be on the left hand side.
 add_add_factor (bool, optional): force introduction aof adjustment term, and an →
 expression to calculate it
 do_preprocess (bool, optional): DESCRIPTION. preprocess the expression
 endo_lhs (bool, optional): If false, accept to normalize for a rhs endogeneous →
 variable
 make_fixable (bool, optional): also make this equation exogenizable
 fitted (bool,optional) : create a fitted equations, without exo and adjustment
 implicit (bool,optional) : This is an implicit frml so to transform to endo__res = →
 (lhs) -(rhs)

preprocessing handels

Returns:
 An instance of the class: Normalized_frml which will contain the different →
 relevant expressions

```

Autoreload keeps the imported module in sync with the source file — convenient while developing `modelnormalize`, not needed if you just use the function.

### 5.4 Basic normalization

The simplest possible case:  $a = b$ . Variable names are upper-cased, and by default an **add factor**  $A_A$  is introduced. `Calc_add_factor` shows the expression used to compute the add factor from historical data, so that the equation tracks the data exactly.

```

normal('a=b')

Endo_var : A
Original : a=b
Preprocessed : A=B
Normalized : A = (B + A_A)
Calc_add_factor : A_A = (A) - (B)

```

With `add_add_factor=0` no adjustment term is introduced — the result is the plain normalized equation.

```
normal('a=b',add_add_factor=0)
```

```
Endo_var : A
Original : a=b
Preprocessed : A=B
Normalized : A = B
```

```
normal('a -c=b',add_add_factor=0)
```

```
Endo_var : A
Original : a -c=b
Preprocessed : A -C=B
Normalized : A = C+ (B)
```

## 5.5 Add factor, fixing and fitted equation combined

All the bells and whistles at once:

- `add_suffix='_AERR'` gives the add factor a custom name (`A_AERR` instead of the default `A_A`),
- `make_fixable=True` makes the equation exogenizable: the dummy `A_D` and the fixing value `A_X` are introduced, so setting `A_D = 1` fixes `A` at the value of `A_X`,
- `make_fitted=True` creates the fitted equation `A_FITTED` — the pure right-hand side without add factor and fixing terms.

```
normal('a=b',add_add_factor=1,add_suffix= '_AERR',make_fitted=True,make_fixable=True)
```

```
Endo_var : A
Original : a=b
Preprocessed : A=B
Normalized : A = (B + A_AERR)* (1 -A_D)+ A_X*A_D
Calc_add_factor : A_AERR = (A) - (B)
Fitted : A_FITTED = B
```

## 5.6 Built-in transformation functions on the right-hand side

The preprocessing step unfolds EViews-style transformation functions into elementary expressions. First `dlog(b)` — the change in the logarithm — which becomes  $\log(B) - \log(B(-1))$ .

```
normal('a=dlog(b)',add_add_factor=1,add_suffix=
'_AERR',make_fitted=True,make_fixable=True)
```

→

```
Endo_var : A
Original : a=dlog(b)
Preprocessed : A=((LOG(B)) -(LOG(B(-1))))
Normalized : A = (((log(B)) -(log(B(-1)))) + A_AERR)* (1 -A_D)+ A_X*A_D
Calc_add_factor : A_AERR = (A) - (((log(B)) -(log(B(-1))))
Fitted : A_FITTED = ((LOG(B)) -(LOG(B(-1))))
```

`logit(b)` is unfolded to  $\log(B/(1 - B))$  — useful when a variable must stay between 0 and 1.

```
normal('a=logit(b)',add_add_factor=1,add_suffix=
'_AERR',make_fitted=True,make_fixable=True)
```

→

```
Endo_var : A
Original : a=logit(b)
Preprocessed : A=(LOG(B/(1.0 -B)))
Normalized : A = ((log(B/(1.0 -B))) + A_AERR)* (1 -A_D)+ A_X*A_D
Calc_add_factor : A_AERR = (A) - ((log(B/(1.0 -B)))
Fitted : A_FITTED = (LOG(B/(1.0 -B)))
```

PCT\_GROWTH(b) is the percentage growth from the previous period:  $100 \cdot (B/B(-1) - 1)$ .

```
normal('a=PCT_GROWTH(b)',add_add_factor=0)
```

```
Endo_var : A
Original : a=PCT_GROWTH(b)
Preprocessed : A= (100 * ((B) / (B(-1)) -1))
Normalized : A = (100 * ((B) / (B(-1)) -1))
```

PCY(b) is the year-over-year percentage growth — the change over four periods, so it assumes quarterly data.

```
normal('a=PCY(b)',add_add_factor=1,add_suffix=
'_AERR',make_fitted=True,make_fixable=True)
```

→

```
Endo_var : A
Original : a=PCY(b)
Preprocessed : A= (100 * ((B) / (B(-4)) -1))
Normalized : A = ((100 * ((B) / (B(-4)) -1)) + A_AERR)* (1 -A_D)+ A_X*A_D
Calc_add_factor : A_AERR = (A) - ((100 * ((B) / (B(-4)) -1)))
Fitted : A_FITTED = (100 * ((B) / (B(-4)) -1))
```

DIFF(b) is the first difference  $B - B(-1)$ .

```
normal('a=DIFF(b)',add_add_factor=1,add_suffix=
'_AERR',make_fitted=True,make_fixable=True)
```

→

```
Endo_var : A
Original : a=DIFF(b)
Preprocessed : A=((B) -(B(-1)))
Normalized : A = (((B) -(B(-1)))) + A_AERR)* (1 -A_D)+ A_X*A_D
Calc_add_factor : A_AERR = (A) - (((B) -(B(-1))))
Fitted : A_FITTED = ((B) -(B(-1)))
```

MOVAVG(b,3) is the moving average over the current and the two preceding periods.

```
normal('a=MOVAVG(b,3)',add_add_factor=1,add_suffix=
'_AERR',make_fitted=True,make_fixable=True)
```

→

```
Endo_var : A
Original : a=MOVAVG(b,3)
Preprocessed : A=((B+B(-1)+B(-2))/3.0)
Normalized : A = (((B+B(-1)+B(-2))/3.0) + A_AERR)* (1 -A_D)+ A_X*A_D
Calc_add_factor : A_AERR = (A) - (((B+B(-1)+B(-2))/3.0))
Fitted : A_FITTED = ((B+B(-1)+B(-2))/3.0)
```

## 5.7 Transformations on the left-hand side

When the endogenous variable enters through a transformation on the left-hand side, **normal** **inverts** the transformation. Here  $\text{PCT\_growth}(a) = n(-1)$  is solved for the level of A.

```
normal('PCT_growth(a) = n(-1)',add_add_factor=0)
```

```
Endo_var : A
Original : PCT_growth(a) = n(-1)
Preprocessed : (100 * ((A) / (A(-1)) -1)) =N(-1)
Normalized : A = (N(-1)) *A(-1)/100+A(-1)
```

Transformation functions can be nested — here a moving average of the percentage change **pct(b)**. Note how the lag operator is distributed over the arguments when the expression is unfolded.

```
normal('a = movavg(pct(b),2)',add_add_factor=0)
```

```
Endo_var : A
Original : a = movavg(pct(b),2)
Preprocessed : A=(((100 * ((B) / (B(-1)) -1)) + (100 * ((B(-1)) / (B(-2))
-1)))/2.0) →
Normalized : A = (((100 * ((B) / (B(-1)) -1)) + (100 * ((B(-1)) / (B(-2))
-1)))/2.0) →
```

`pct_growth` on **both** sides: the left-hand side is inverted to isolate `C`, while the right-hand side is simply unfolded.

Note: the right-hand variable is called `DD` because `D` is reserved for the EViews difference operator `D(...)` — once the equation is unfolded, a lag like `D(-1)` would be indistinguishable from a call to the operator, so `normal` raises an error asking you to rename such a variable.

```
normal('pct_growth(c) = pct_growth(dd)',add_add_factor=0)
```

```
Endo_var : C
Original : pct_growth(c) = pct_growth(dd)
Preprocessed : (100 * ((C) / (C(-1)) -1)) = (100 * ((DD) / (DD(-1)) -1))
Normalized : C = ((100 * ((DD) / (DD(-1)) -1))) *C(-1)/100+C(-1)
```

A variable can not share its name with one of the reserved function names (`D`, `DIFF`, `DLOG`, `PCT_GROWTH`, `PCY`, `LOGIT`, `MOVAVG`): as soon as such a variable appears with a lag — like `DIFF(-1)` — it is indistinguishable from a function call, and `normal` raises an explicit error asking you to rename it. In the two cells below the exception is caught just to display the message.

First a variable named `DIFF`, lagged directly by the user:

```
try:
 normal('a = diff(b) + diff(-1)',add_add_factor=0)
except Exception as e:
 print('Exception raised')
 print(e)
```

```
Exception raised
A variable can not be named DIFF, as DIFF(is a function in the business language.
DIFF(-1) looks like a lagged variable DIFF. Rename the variable.
Original expression: a=diff(b)+diff(-1)
Processed so far : A=((B) -(B(-1)))+DIFF(-1)
```

And a variable named `D` — here the offending lag `D(-1)` is not written by the user but *created* by the unfolding of `pct_growth(d)`, as the “Processed so far” line shows:

```
try:
 normal('pct_growth(c) = pct_growth(d)',add_add_factor=0)
except Exception as e:
 print('Exception raised')
 print(e)
```

```
Exception raised
A variable can not be named D, as D(is a function in the business language.
D(-1) looks like a lagged variable D. Rename the variable.
Original expression: pct_growth(c)=pct_growth(d)
Processed so far : (100 * ((C) / (C(-1)) -1)) = (100 * ((D) / (D(-1)) -1))
```

When the left-hand side is a transformation and an add factor is requested (the default), the add factor enters in the **transformed space**: here `C_A` is measured in percentage growth, and `Calc_add_factor` computes it accordingly.

```
normal('pct_growth(c) = z+pct(b) + pct(e)')
```

```

Endo_var : C
Original : pct_growth(c) = z+pct(b) + pct(e)
Preprocessed : (100 * ((C) / (C(-1)) -1)) =Z+ (100 * ((B) / (B(-1)) -1)) +
(100 * ((E) / (E(-1)) -1))
Normalized : C = C_A*C(-1)/100+ (Z+ (100 * ((B) / (B(-1)) -1)) + (100 * ((E)
/ (E(-1)) -1))) *C(-1)/100+C(-1)
Calc_add_factor : C_A = 100*C/C(-1) - ((Z+ (100 * ((B) / (B(-1)) -1)) + (100 * (
(E) / (E(-1)) -1)))) -100

```

## 5.8 A real-world example

An estimated error-correction equation from a World Bank EViews model. DLOG on the left-hand side gives a **multiplicative** normalization through EXP(...), and the add factor works on the log change. EViews-specific pieces such as @DURING("2011") pass through the preprocessing. The .fprint attribute prints the result without truncation — handy for long equations.

```

normal("DLOG(SAUNECONGOVTXN) = -0.323583422052*(LOG(SAUNECONGOVTXN(-1))
-GOVSHAREWB*LOG(SAUNEYWRPGOVCN(-1)) -(1 -GOVSHAREWB)*LOG(SAUNECONPRVTXN(
-1)))+0.545415878897*DLOG(SAUNECONGOVTXN(-1))+(1
-0.545415878897)*(GOVSHAREWB)*DLOG(SAUNEYWRPGOVCN) +(1 -0.545415878897)*(1
-GOVSHAREWB)*DLOG(SAUNECONPRVTXN) -1.56254616684
-0.0613991001064*@DURING("2011"))".fprint

```

```

Endo_var : SAUNECONGOVTXN
Original : DLOG(SAUNECONGOVTXN) = -0.323583422052*(LOG(SAUNECONGOVTXN(-1))
-GOVSHAREWB*LOG(SAUNEYWRPGOVCN(-1)) -(1 -GOVSHAREWB)*LOG(SAUNECONPRVTXN(
-1)))+0.545415878897*DLOG(SAUNECONGOVTXN(-1))+(1
-0.545415878897)*(GOVSHAREWB)*DLOG(SAUNEYWRPGOVCN) +(1 -0.545415878897)*(1
-GOVSHAREWB)*DLOG(SAUNECONPRVTXN) -1.56254616684 -0.0613991001064*@DURING(2011)
Preprocessed : ((LOG(SAUNECONGOVTXN)) -(LOG(SAUNECONGOVTXN(-1))))=
-0.323583422052*(LOG(SAUNECONGOVTXN(-1)) -GOVSHAREWB*LOG(SAUNEYWRPGOVCN(-1)) -(1
-GOVSHAREWB)*LOG(SAUNECONPRVTXN(-1)))+0.545415878897*((LOG(SAUNECONGOVTXN(-1)))
-(LOG(SAUNECONGOVTXN(-2))))+(1 -0.545415878897)*(GOVSHAREWB)*((LOG(SAUNEYWRPGOVCN))
-(LOG(SAUNEYWRPGOVCN(-1))))+(1 -0.545415878897)*(1
-GOVSHAREWB)*((LOG(SAUNECONPRVTXN)) -(LOG(SAUNECONPRVTXN(-1)))) -1.56254616684
-0.0613991001064*@DURING(+2011)
Normalized : SAUNECONGOVTXN = SAUNECONGOVTXN(-1)*EXP(SAUNECONGOVTXN_A+ (
-0.323583422052*(LOG(SAUNECONGOVTXN(-1)) -GOVSHAREWB*LOG(SAUNEYWRPGOVCN(-1)) -(1
-GOVSHAREWB)*LOG(SAUNECONPRVTXN(-1)))+0.545415878897*((LOG(SAUNECONGOVTXN(-1)))
-(LOG(SAUNECONGOVTXN(-2))))+(1 -0.545415878897)*(GOVSHAREWB)*((LOG(SAUNEYWRPGOVCN))
-(LOG(SAUNEYWRPGOVCN(-1))))+(1 -0.545415878897)*(1
-GOVSHAREWB)*((LOG(SAUNECONPRVTXN)) -(LOG(SAUNECONPRVTXN(-1)))) -1.56254616684
-0.0613991001064*@DURING(+2011))
Calc_add_factor : SAUNECONGOVTXN_A = - ((-0.323583422052*(LOG(SAUNECONGOVTXN(-1))
-GOVSHAREWB*LOG(SAUNEYWRPGOVCN(-1)) -(1 -GOVSHAREWB)*LOG(SAUNECONPRVTXN(
-1)))+0.545415878897*((LOG(SAUNECONGOVTXN(-1))) -(LOG(SAUNECONGOVTXN(-2))))+(1
-0.545415878897)*(GOVSHAREWB)*((LOG(SAUNEYWRPGOVCN)) -(LOG(SAUNEYWRPGOVCN(-1))))+(1
-0.545415878897)*(1 -GOVSHAREWB)*((LOG(SAUNECONPRVTXN)) -(LOG(SAUNECONPRVTXN(-1))))
-1.56254616684 -0.0613991001064*@DURING(+2011))) +LOG(SAUNECONGOVTXN)
-LOG(SAUNECONGOVTXN(-1))

```

DIFF on the left-hand side is inverted additively:  $A = \text{rhs} + A(-1)$ .

```
normal("Diff(a) = b")
```

```

Endo_var : A
Original : Diff(a) = b
Preprocessed : ((A) -(A(-1)))=B
Normalized : A = A_A+ (B) +A(-1)
Calc_add_factor : A_A = A - ((B)) -A(-1)

```

## 5.9 Leads and the EViews D() operator

The EViews difference operator  $D(x, n, s)$  is also handled, and expressions may contain **leads** such as  $QLHP(+1)$ .

```
normal('a = D(LOG(QLHP(+1)), 0, 1)')
```

```
Endo_var : A
Original : a = D(LOG(QLHP(+1)), 0, 1)
Preprocessed : A=((LOG(QLHP(+1))) -(LOG(QLHP)))
Normalized : A = (((log(QLHP(+1))) -(log(QLHP))) + A_A)
Calc_add_factor : A_A = (A) - (((log(QLHP(+1))) -(log(QLHP))))
```

$D(x)$  with default arguments is a plain first difference — the result is the same as above.

```
normal('a = D(LOG(QLHP(+1)))')
```

```
Endo_var : A
Original : a = D(LOG(QLHP(+1)))
Preprocessed : A=((LOG(QLHP(+1))) -(LOG(QLHP)))
Normalized : A = (((log(QLHP(+1))) -(log(QLHP))) + A_A)
Calc_add_factor : A_A = (A) - (((log(QLHP(+1))) -(log(QLHP))))
```

## 5.10 Choosing the endogenous variable

By default the first variable on the left-hand side becomes the endogenous variable. With **the\_endo** and **endo\_lhs=False** the equation can instead be solved for a variable on the **right-hand side** — here  $F$ . Combined with **make\_fixable=True** the resulting equation is also exogenizable.

```
normal('a = gamma+ f+0',the_endo='f',endo_lhs=False,make_fixable =True)
```

```
Endo_var : F
Original : a = gamma+ f+0
Preprocessed : A=GAMMA+F+0
Normalized : F = (A -F_A -GAMMA -0) * (1 -F_D)+ F_X*F_D
Calc_add_factor : F_A = A -F -GAMMA -0
```

## 5.11 Implicit equations

Some equations — typically equilibrium conditions — cannot be solved analytically for the endogenous variable. With **implicit=True** the equation is recast as a **residual equation**:  $PRICE\_RES = supply - demand$ , which is driven to zero by adjusting the variable named in **the\_endo** (here  $PRICE$ ).

Note that a model containing implicit equations has to be solved with one of the **Newton-type solvers** — the Gauss-Seidel type solvers require every equation to be normalized with the endogenous variable alone on the left-hand side, which is exactly what an implicit equation lacks.

```
normal('demand = supply',the_endo='price',implicit=True)
```

```
Endo_var : PRICE
Original : demand = supply
Preprocessed : DEMAND=SUPPLY
Normalized : PRICE__RES = (SUPPLY) - (DEMAND)
Un_normalized : PRICE__RES = (SUPPLY) - (DEMAND)
```

## 5.12 Summary: the preprocessing steps

The table lists the preprocessing steps in the order they are applied. Each rewrite is repeated until no occurrence is left, so the functions can be nested — `movavg(pct(b),2)` for instance is unfolded completely.

| Step | Pattern               | Rewritten as                    | Meaning                                             |
|------|-----------------------|---------------------------------|-----------------------------------------------------|
| 1    | <i>whole equation</i> | upper-cased, blanks removed     | housekeeping                                        |
| 2    | PCT(x)                | PCT_GROWTH(x)                   | alias — handled in step 6                           |
| 3    | DLOG(x)               | DIFF(LOG(x))                    | change in the logarithm — handled further in step 8 |
| 4    | LOGIT(x)              | (LOG(x/(1.0 - x)))              | log-odds of a variable between 0 and 1              |
| 5    | MOVAVG(x,n)           | ((x + x(-1) + ... + x(-n+1))/n) | moving average over n periods                       |
| 6    | PCT_GROWTH(x)         | (100*((x)/(x(-1)) - 1))         | percentage growth from the previous period          |
| 7    | PCY(x)                | (100*((x)/(x(-4)) - 1))         | year-over-year percentage growth (quarterly data)   |
| 8    | DIFF(x)               | ((x) - (x(-1)))                 | first difference                                    |
| 9    | D(x) or D(x,0,1)      | ((x) - (x(-1)))                 | EViews difference operator                          |

Before every rewrite the argument is checked: a pure signed integer like `DIFF(-1)` must be a *lagged variable* named like a function, and an explicit error is raised (see the section on reserved names above).

## Chapter 6

# When using Makemodel og %%Makemymodel the tag <add>will introduce an add factor

In this we will look into some of the behavior of this

### 6.1 Activate the %%Makemymodel magic cell

This is a convinient wrapper around the Makemodel function

```
| import modeljupytermagic # ti activate the %% %%Makemymodel magic cell
```

### 6.2 A simple equation

#### 6.2.1 When the equation is already normalized

That is the left hand side is a variable

```
| %%Makemymodel test show
| > a = b
|
| ```text
| > a = b
| ```
|
| FRML <> A = B $
|
| %%Makemymodel test show
| ><add> a = b
|
| ```text
| ><add> a = b
| ```
|
| FRML <ADD> A = (B + A_A) $
|
| FRML <CALC_ADD_FACTOR> A_A = (A) - (B) $
|
| %%Makemymodel test show
| ><add> a = log(b)
|
| ```text
| ><add> a = log(b)
| ```
|
| FRML <ADD> A = (log(B) + A_A) $
|
| FRML <CALC_ADD_FACTOR> A_A = (A) - (log(B)) $
```

## 6.2.2 When the equation has to be normalized no differences on left hand side

That is the left hand side is a variable

```
%%Makemymodel test show
><add> log(a) = b

```text
><add> log(a) = b
```

FRML <ADD> A = EXP(A_A+ (B)) $

FRML <CALC_ADD_FACTOR> A_A = - ((B)) +LOG(A) $

%%Makemymodel test show
><add> log(a) = diff(b)

```text
><add> log(a) = diff(b)
```

FRML <ADD> A = EXP(A_A+ (((B) -(B(-1))))) $

FRML <CALC_ADD_FACTOR> A_A = - (((B) -(B(-1)))) +LOG(A) $

%%Makemymodel test show
><add> log(a)+d+e = b

```text
><add> log(a)+d+e = b
```

FRML <ADD> A = EXP(A_A -D -E+ (B)) $

FRML <CALC_ADD_FACTOR> A_A = D+E - ((B)) +LOG(A) $
```

## 6.2.3 When the equation has to be normalized have differences on left hand side

That is the left hand side is a variable

```
%%Makemymodel test show
><add> diff(a) = b

```text
><add> diff(a) = b
```

FRML <ADD> A = A_A+ (B) +A(-1) $

FRML <CALC_ADD_FACTOR> A_A = A - ((B)) -A(-1) $

%%Makemymodel test show
><add, exo,fit> dlog(a) = b

```text
><add, exo,fit> dlog(a) = b
```

FRML <ADD, EXO,FIT> A = (A(-1)*EXP(A_A+ (B))) * (1 -A_D)+ A_X*A_D $
FRML <FIT> A_FITTED = A(-1)*EXP((B)) $
FRML <CALC_ADD_FACTOR> A_A = - ((B)) +LOG(A) -LOG(A(-1)) $
```

```
%%Makemymodel test show
><add, exo,fit> a = b *c
```

```
```text
><add, exo,fit> a = b *c
```
```

```
FRML <ADD, EXO,FIT> A = (B*C + A_A)* (1 -A_D)+ A_X*A_D $
FRML <FIT> A_FITTED = B*C $
FRML <CALC_ADD_FACTOR> A_A = (A) - (B*C) $
```